

ESCUELA DE INGENIERÍA INFORMÁTICA DE
VALLADOLID

UNIVERSIDAD DE VALLADOLID

Prácticas ESI-ERSS

Pruebas de Carga y Monitorización



Universidad de Valladolid

Grupo 02

Autores:

Alfredo del Val Ramos

Daniel Garcia Salinas

Francisco Ivan San Segundo Alvarez

Gabriel Ferrero Herrera

Asignatura:

Evaluación de Sistemas Informáticos - Evaluación y Rendimiento de Sistemas Software

3 de junio de 2025

Índice

1. Introducción	2
2. Implementación de la Práctica	3
2.1. Estructura del Proyecto	3
2.2. Base de Datos	4
2.2.1. users	4
2.2.2. products	4
2.2.3. cart_items	5
2.2.4. orders	5
2.2.5. order_items	6
2.3. Códigos Realizados	6
2.4. __init__.py	6
2.4.1. auth.py	9
2.4.2. cart.py	11
2.4.3. menu.py	14
2.4.4. orders.py	15
2.4.5. products.py	18
2.4.6. Script de Pruebas de Carga con Locust(<code>locustfile.py</code>)	21
2.4.7. HTML y CSS	24
2.4.8. Plantillas HTML y Lógica del Cliente	25
3. Resultados y Análisis de las Pruebas	29
3.1. Pruebas de carga de Grafana	29
3.1.1. Prueba 1	29
3.1.2. Prueba 2	31
3.1.3. Prueba 3	32
3.1.4. Prueba 4	33
3.1.5. Prueba 5	35
3.1.6. Prueba 6	36
3.2. Justificación del fallo de las pruebas de carga	37
4. Métricas del Locust	38
4.1. Número de requests	38
4.2. Tiempo medio de respuesta	41
5. Métricas de Modelo de Negocio	44
5.1. Número de pedidos	44
5.2. Productos	45
5.3. Productos por pedido	45
5.4. Productos por carrito	46
5.5. Número de usuarios	46
6. Conclusiones	47
7. Referencias	49

1. Introducción

Este informe detalla la realización de la segunda práctica de la asignatura de Evaluación de Sistemas Informáticos (ESI). El objetivo principal de la práctica ha sido la realización de una página web, para su posterior evaluación de rendimiento, usando el software de Locust y Grafana, que nos permiten proveer de carga al sistema y realizar las mediciones pertinentes para comprobar el rendimiento final.

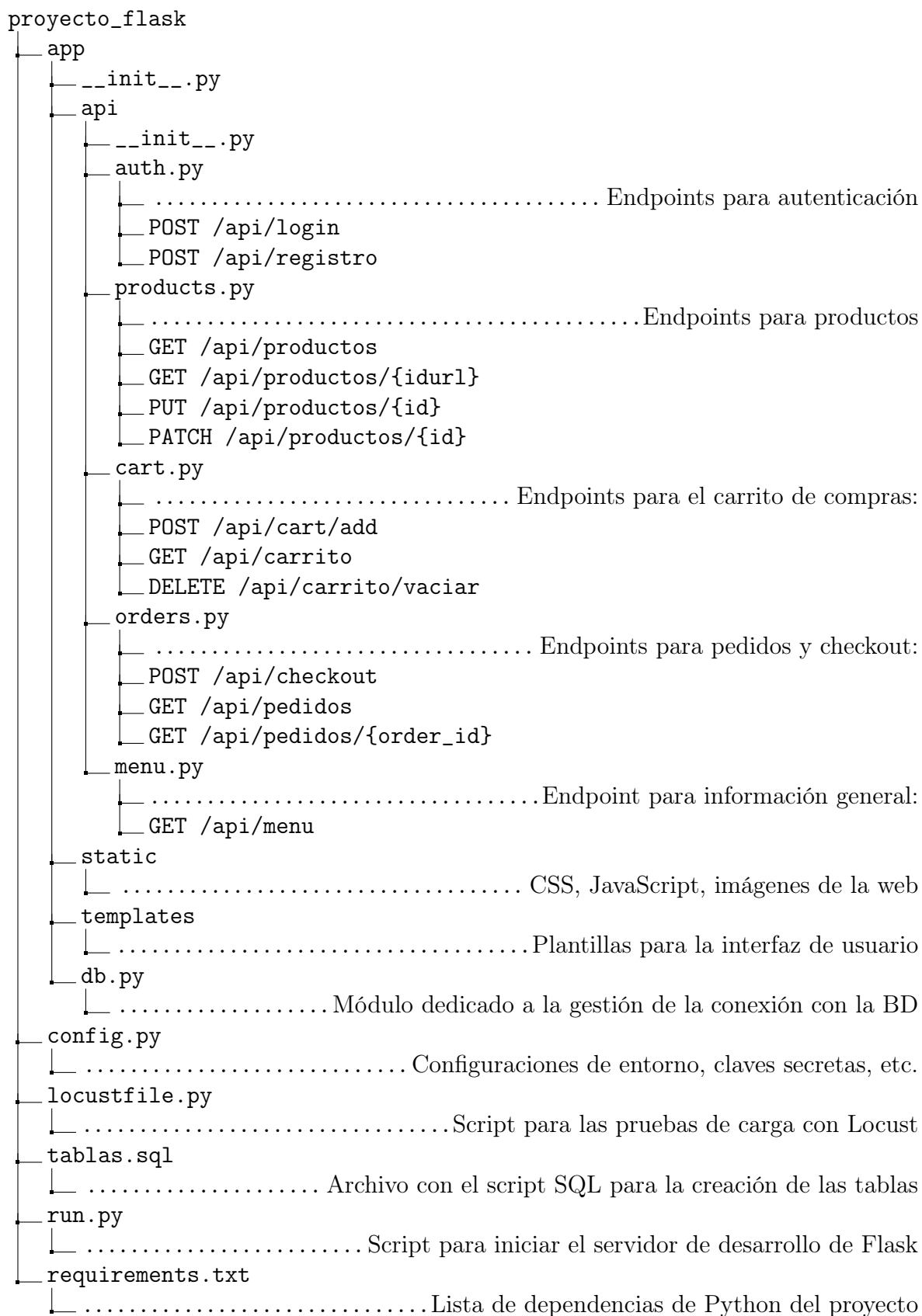
El proyecto se desarrolló de la siguiente forma:

- Inicialmente, se procedió con la realización de una página web, denominada Arte Visual. Esta es una tienda simple especializada en la venta de 5 pósters de edición limitada. Para su desarrollo, se utilizó el framework flask en Python, incluyendo una interfaz de programación de aplicaciones (API) Restful. La persistencia para la base de datos se gestionó con MariaDB, cuyo esquema, presente más adelante, se diseñó para soportar la funcionalidad requerida por la página web. Tras comprobar que los endpoints funcionaban correctamente, se procedió a la realización de código html y css, que permite el fácil acceso y uso de los distintos recursos de la página web.
- Tras desplegar la página "*ArteVisual*", se diseñaron y ejecutaron scripts de prueba con Locust. Estos, consiguen simular comportamientos de usuarios, permitiendo añadir esa carga al sistema que queremos medir.
- Por último, continuando con la primera de las prácticas de esta asignatura, se observaron las gráficas ya creadas y se crearon otras nuevas, que nos permitieron observar como la carga de los diferentes usuarios afectaba al rendimiento de nuestra máquina virtual.

Esta introducción expone de forma breve el conjunto de la práctica. Cada parte realizada se irá explicando más a fondo en su respectivo punto, desarrollando cada tema y detallando el porque de muchas de las funcionalidades descritas.

2. Implementación de la Práctica

2.1. Estructura del Proyecto



Esta estructura se ha establecido ya que permite una estructura de directorios modular y bien definida, siguiendo los patrones comunes del desarrollo con Flask. Mediante el uso de paquetes y de Blueprints, creamos una estructura escalable y robusta.

2.2. Base de Datos

Para esta práctica se ha diseñado y creado una base de datos relacional en MariaDB para albergar los datos de negocio de la aplicación web "*ArteVisual*". La base de datos consta de cinco tablas principales, cuyas definiciones y propósitos se detallan a continuación:

2.2.1. users

Almacena la información de los usuarios registrados en la plataforma, incluyendo sus credenciales de acceso y el tipo de usuario.

```
1 CREATE TABLE 'users' (  
2   'id' int(11) NOT NULL AUTO_INCREMENT,  
3   'name' varchar(255) NOT NULL,  
4   'email' varchar(255) NOT NULL,  
5   'password_hash' varchar(255) NOT NULL,  
6   'created_at' timestamp NOT NULL DEFAULT current_timestamp(),  
7   'user_type' varchar(10) NOT NULL DEFAULT 'not_admin',  
8   PRIMARY KEY ('id'),  
9   UNIQUE KEY 'email' ('email'),  
10  CONSTRAINT 'chk_user_type' CHECK ('user_type' in ('admin',  
11    not_admin'))  
12 )
```

Listing 1: Definición de la tabla users

Propósito: Esta tabla es fundamental para la gestión de cuentas, permitiendo el registro, inicio de sesión y la posible diferenciación de roles (administrador vs. usuario normal) dentro de la aplicación.

2.2.2. products

Contiene el catálogo de los productos (pósters) ofrecidos en la tienda, con detalles como título, descripción, precio, stock disponible e identificador para URL.

```
1 CREATE TABLE 'products' (  
2   'id' int(11) NOT NULL AUTO_INCREMENT,  
3   'idurl' varchar(100) NOT NULL,  
4   'title' varchar(255) NOT NULL,  
5   'description' text DEFAULT NULL,  
6   'artist' varchar(100) DEFAULT NULL,  
7   'price' decimal(10,2) NOT NULL,  
8   'stock' int(11) NOT NULL,  
9   'image_url' varchar(512) NOT NULL,  
10  'created_at' timestamp NOT NULL DEFAULT current_timestamp(),  
11  'updated_at' timestamp NOT NULL DEFAULT current_timestamp() ON  
12  UPDATE current_timestamp(),  
13  PRIMARY KEY ('id'),
```

```

13  UNIQUE KEY 'idurl' ('idurl'),
14  CONSTRAINT 'chk_product_price' CHECK ('price' >= 0),
15  CONSTRAINT 'chk_product_stock' CHECK ('stock' >= 0)
16 )

```

Listing 2: Definición de la tabla `products`

Propósito: Sirve como el inventario digital de la tienda, permitiendo la visualización de los artículos y la gestión de su disponibilidad y precios.

2.2.3. `cart_items`

Representa los artículos que un usuario ha añadido a su carrito de compras, vinculando al usuario, el producto y la cantidad seleccionada.

```

1  CREATE TABLE 'cart_items' (
2    'id' int(11) NOT NULL AUTO_INCREMENT,
3    'user_id' int(11) NOT NULL,
4    'product_id' int(11) NOT NULL,
5    'quantity' int(10) unsigned NOT NULL DEFAULT 1,
6    'added_at' timestamp NOT NULL DEFAULT current_timestamp(),
7    PRIMARY KEY ('id'),
8    KEY 'fk_cart_product' ('product_id'),
9    KEY 'idx_cart_user_id' ('user_id'),
10   CONSTRAINT 'fk_cart_product' FOREIGN KEY ('product_id')
11     REFERENCES 'products' ('id') ON DELETE CASCADE,
12   CONSTRAINT 'fk_cart_user' FOREIGN KEY ('user_id') REFERENCES '
    users' ('id') ON DELETE CASCADE

```

Listing 3: Definición de la tabla `cart_items`

Propósito: Esta tabla es esencial para la funcionalidad del carrito de compras, permitiendo a los usuarios acumular productos antes de proceder al pago. Las claves foráneas aseguran la integridad referencial con las tablas de usuarios y productos.

2.2.4. `orders`

Registra la información general de cada pedido completado por un usuario, incluyendo el importe total, el estado del pedido y la fecha de creación.

```

1  CREATE TABLE 'orders' (
2    'id' int(11) NOT NULL AUTO_INCREMENT,
3    'user_id' int(11) NOT NULL,
4    'total_amount' decimal(10,2) NOT NULL,
5    'status' varchar(50) NOT NULL DEFAULT 'completed',
6    'created_at' timestamp NOT NULL DEFAULT current_timestamp(),
7    PRIMARY KEY ('id'),
8    KEY 'fk_order_user' ('user_id'),
9    CONSTRAINT 'fk_order_user' FOREIGN KEY ('user_id') REFERENCES '
10     users' ('id'),
11   CONSTRAINT 'chk_order_total' CHECK ('total_amount' >= 0)

```

Listing 4: Definición de la tabla `orders`

Propósito: Funciona como el registro maestro de todas las transacciones de venta realizadas, vinculando cada pedido a un usuario.

2.2.5. order_items

Detalla los productos específicos, cantidades y precios al momento de la compra para cada pedido registrado en la tabla `orders`.

```
1 CREATE TABLE 'order_items' (  
2   'id' int(11) NOT NULL AUTO_INCREMENT,  
3   'order_id' int(11) NOT NULL,  
4   'product_id' int(11) NOT NULL,  
5   'quantity' int(10) unsigned NOT NULL,  
6   'price_at_purchase' decimal(10,2) NOT NULL,  
7   PRIMARY KEY ('id'),  
8   KEY 'fk_orderitem_order' ('order_id'),  
9   KEY 'fk_orderitem_product' ('product_id'),  
10  CONSTRAINT 'fk_orderitem_order' FOREIGN KEY ('order_id')  
    REFERENCES 'orders' ('id') ON DELETE CASCADE,  
11  CONSTRAINT 'fk_orderitem_product' FOREIGN KEY ('product_id')  
    REFERENCES 'products' ('id'),  
12  CONSTRAINT 'chk_orderitem_quantity' CHECK ('quantity' > 0),  
13  CONSTRAINT 'chk_orderitem_price' CHECK ('price_at_purchase' >= 0)  
14 )
```

Listing 5: Definición de la tabla `order_items`

Propósito: Esta tabla de unión es necesaria para mantener un histórico preciso del contenido de cada pedido, incluyendo el precio de los productos en el momento exacto de la compra, independientemente de cambios futuros en el catálogo.

2.3. Códigos Realizados

A continuación, mostramos los distintos códigos para la realización de la práctica. Debido a su extensión, no mostraremos el código al completo ya que este se proporcionará al profesor por otro método. Simplemente indicaremos para cada código, puntos importantes y decisiones de implementación que hayamos tomado.

2.4. __init__.py

Este archivo es el principal en el paquete de la aplicación Flask (`app/`). Su función principal es inicializar y configurar la aplicación. Utiliza el patrón Application Factory (`create_app`) para crear la instancia de la aplicación, carga la configuración específica del entorno, gestiona la conexión a la base de datos para cada solicitud, registra todos los Blueprints que definen las rutas de la API y define algunas rutas directas para servir las páginas HTML de la interfaz de usuario y rutas de prueba.

A continuación, se analizan las partes más relevantes del código de `__init__.py`:

Gestión de la Conexión a la Base de Datos

La aplicación gestiona la conexión a la base de datos MariaDB de forma eficiente por solicitud.

```

1 def get_db():
2     if 'db' not in g:
3         try:
4             g.db = pymysql.connect(
5                 host=current_app.config['DB_HOST'],
6                 user=current_app.config['DB_USER'],
7                 password=current_app.config['DB_PASSWORD'],
8                 database=current_app.config['DB_NAME'],
9                 port=current_app.config['DB_PORT'],
10                cursorclass=pymysql.cursors.DictCursor
11            )
12        except pymysql.Error as e:
13            print(f"Error conectando a MariaDB: {e}")
14            g.db = None
15    return g.db

```

Listing 6: Obtención de conexión a la BD en `__init__.py`

Explicación: La función `get_db` establece una conexión a la base de datos si aún no existe una para la solicitud actual, almacenándola en el objeto global de aplicación `g` de Flask. Utiliza `pymysql` y configura el cursor para devolver filas como diccionarios. La configuración de la conexión se obtiene del objeto de configuración de la aplicación actual (`current_app.config`).

```

17 def close_db(e=None):
18     db = g.pop('db', None)
19     if db is not None:
20         db.close()

```

Listing 7: Cierre de conexión a la BD en `__init__.py`

Explicación: La función `close_db` se encarga de cerrar la conexión a la base de datos al finalizar la solicitud, y se registra para ser llamada automáticamente por Flask.

Creación y Configuración de la Aplicación

```

22 def create_app(config_name='default'):
23     app = Flask(__name__)
24     app.config.from_object(config_by_name[config_name])
25     app.teardown_appcontext(close_db)
26     # ... (registro de blueprints y rutas omitido por brevedad) ...
27     return app

```

Listing 8: Función factory `create_app` en `__init__.py`

Explicación: La función `create_app` construye la instancia de la aplicación Flask (`app`), carga la configuración desde un objeto (probablemente definido en `config.py`) y registra la función `close_db` para que se ejecute al final de cada contexto de solicitud.

Registro de Blueprints para la API

Los diferentes módulos que componen la API RESTful se organizan mediante **Blueprints**, los cuales se registran en la aplicación con el prefijo `/api`.

```

33     from .api import auth
34     app.register_blueprint(auth.auth_bp, url_prefix='/api')
35
36     from .api import products
37     app.register_blueprint(products.products_bp, url_prefix='/api')
38
39     from .api import cart
40     app.register_blueprint(cart.cart_bp, url_prefix='/api')
41
42     from .api import orders
43     app.register_blueprint(orders.orders_bp, url_prefix='/api')
44
45     from .api import menu
46     app.register_blueprint(menu.menu_bp, url_prefix='/api')

```

Listing 9: Registro de Blueprints de la API en `__init__.py`

Explicación: Cada módulo de la API (auth, products, etc.), que encapsula la lógica para un conjunto de endpoints, se importa y registra como un Blueprint. El prefijo /api asignado globalmente aquí asegura que todas las rutas definidas dentro de estos blueprints sean accesibles bajo /api/nombre-ruta-blueprint/....

Definición de Rutas para la Interfaz de Usuario (HTML)

Se definen rutas directas para servir las páginas HTML de la tienda.

```

48     @app.route('/registro')
49     def registro_page():
50         return render_template('crearCuenta.html', page_title="Crear
51         Cuenta")
52
53     @app.route('/login')
54     def login_page():
55         return render_template('inicioSesion.html', page_title="
56         Inicio de Sesion")

```

Listing 10: Ejemplo de ruta para servir una página HTML en `__init__.py`

Explicación: `render_template` se usa para generar y devolver las páginas HTML correspondientes, formando la interfaz visible para el usuario en un navegador.

Rutas de Prueba y Diagnóstico

```

81     @app.route('/db_test')
82     def db_test():
83         db_conn = get_db()
84         if db_conn is None:
85             return jsonify({"status": "error", "message": "No se
86             pudo conectar a la base de datos"}), 500
87         try:
88             with db_conn.cursor() as cursor:
89                 cursor.execute("SELECT VERSION()")

```

```

89         version = cursor.fetchone()
90         return jsonify({"status": "success", "db_version":
version})
91     except pymysql.Error as e:
92         return jsonify({"status": "error", "message": f"Error
en consulta: {e}"}), 500

```

Listing 11: Ruta de prueba de conexión a la BD en `__init__.py`

Explicación: La ruta `/db_test` permite una verificación rápida de la conectividad con la base de datos, devolviendo la versión de la misma como confirmación.

2.4.1. `auth.py`

Este módulo es responsable de la autenticación y el registro de usuarios en la aplicación. Define los endpoints de la API necesarios para que los usuarios puedan iniciar sesión y crear nuevas cuentas. El inicio de sesión gestiona la creación de una sesión de Flask, mientras que el registro se encarga de la validación y almacenamiento de nuevos usuarios, incluyendo el hashing seguro de contraseñas.

Definición del Blueprint Al igual que otros módulos de la API, `auth.py` define un Blueprint para agrupar sus rutas específicas.

```

1 auth_bp = Blueprint('auth', __name__)

```

Listing 12: Creación del Blueprint `auth_bp` en `auth.py`

Explicación: Se define `auth_bp`. Este Blueprint se registra en el archivo `__init__.py` (ver Subsección 9) con el prefijo `/api`, haciendo que las rutas aquí definidas sean accesibles bajo, por ejemplo, `/api/login`.

Endpoint de Inicio de Sesión (POST `/api/login`) Este endpoint maneja las solicitudes de inicio de sesión, esperando credenciales en formato JSON y gestionando la sesión del usuario.

```

4 @auth_bp.route('/login', methods=['GET', 'POST'])
5 def check_login():
6     if request.method == 'POST':
7         if not request.is_json:
8             return jsonify({"success": False, "message": "Petición
9             invalida: Se esperaba un json"}), 400
10         data = request.get_json()
11         email_form=data.get('username') # Espera 'username' como
12         clave para el email
13         contrasena_form=data.get('password')
14
15         db=get_db()
16
17         try:
18             cursor=db.cursor()
19             sql= "SELECT id,email,password_hash,user_type from
20             users where email = %s"
21             cursor.execute(sql,(email_form,))

```

```

19         user_data= cursor.fetchone()
20         # Cierre de cursor omitido
21     except Exception as e:
22         return jsonify({"success": False, "message": "Error en
consulta"}), 500
23
24     if user_data is None:
25         return jsonify({"success": False, "message": "Email no
registrado."}), 401
26     else:
27         if check_password_hash(user_data['password_hash'],
contrasena_form):
28             session.clear()
29             session['email']=user_data['email']
30             session['user_id'] = user_data['id']
31             session['user_type'] = user_data['user_type']
32             return jsonify({"success": True, "message": "Login
exitoso!"}), 200
33         else:
34             return jsonify({"success": False, "message": "
Contrasena incorrecta."}), 401
35     if request.method == 'GET':
36         return redirect(url_for('login_page'))

```

Listing 13: Lógica principal del endpoint POST /api/login

Explicación: El endpoint /api/login procesa peticiones POST. Primero, verifica que la solicitud contenga un cuerpo JSON y extrae el email (bajo la clave 'username') y la contraseña (bajo la clave 'password'). Utiliza la función `get_db` (descrita en la Subsección 6 para la gestión de la base de datos en `__init__.py`). Se consulta la base de datos para encontrar al usuario por su email. Si el usuario existe, se verifica la contraseña proporcionada contra el hash almacenado usando `check_password_hash`. En caso de éxito, se limpia la sesión actual y se almacenan los datos relevantes del usuario (`user_id`, `email`, `user_type`) en el objeto `session` de Flask, devolviendo una respuesta JSON afirmativa. Los fallos resultan en respuestas JSON con códigos de error 400 o 401. Las peticiones GET a esta ruta son redirigidas a la página de login HTML (definida en `__init__.py`, ver Subsección 10).

Endpoint de Registro de Nuevos Usuarios (POST /api/registro) Este endpoint maneja la creación de nuevas cuentas de usuario, esperando datos de un formulario HTML.

```

60 @auth_bp.route('/registro', methods=['GET', 'POST'])
61 def check_registro():
62     if request.method == 'POST':
63         usuario_form=request.form.get('usuario')
64         contrasena_form=request.form.get('contrasena')
65         contrasena_hashed=generate_password_hash(contrasena_form)
66         email_form=request.form.get('email')
67
68         db=get_db()
69         # Ejemplo de consulta: sql="SELECT id from users where
email=%s"

```

```

70         if user_data: # Si el email ya existe
71             flash("El email introducido ya esta registrado...", '
72             danger')
73         return redirect(url_for('registro_page'))
74     else:
75         try:
76             cursor=db.cursor()
77             sql_insert="INSERT into users(name,email,
78             password_hash) VALUES (%s,%s,%s)"
79             valores=(usuario_form,email_form,contrasena_hashed
80             )
81             cursor.execute(sql_insert,valores)
82             db.commit()
83         except Exception as e:
84             flash("Error al registrar usuario", 'danger')
85             return redirect(url_for('registro_page'))
86         return redirect(url_for('login_page'))
87     return redirect(url_for('registro_page'))

```

Listing 14: Lógica principal del endpoint POST /api/registro

Explicación: El endpoint /api/registro procesa peticiones POST provenientes de un formulario HTML (utilizando `request.form`). Recoge el nombre de usuario, email y contraseña. La contraseña se hashea usando `generate_password_hash` antes de cualquier operación de base de datos. Se verifica si el email ya está registrado. Si no lo está, se inserta el nuevo usuario en la tabla `users`. Este endpoint utiliza mensajes `flash` y redirecciones a las páginas HTML de registro o login (definidas en `__init__.py`, ver Subsección 10), indicando que está diseñado principalmente para la interacción a través de un navegador web y no como un endpoint API JSON puro para el registro.

2.4.2. cart.py

Este módulo gestiona toda la funcionalidad relacionada con el carrito de compras de los usuarios. Permite a los usuarios autenticados añadir productos a su carrito, visualizar su contenido actual (incluyendo el cálculo del precio total) y vaciarlo completamente. Todos los endpoints de este módulo requieren que el usuario haya iniciado sesión previamente.

Definición del Blueprint y Decorador de Autenticación Se establece un Blueprint para las rutas del carrito y un decorador para protegerlas.

```

1 cart_bp = Blueprint('cart', __name__)
2
3 def login_required(f):
4     @wraps(f)
5     def decorated_function(*args, **kwargs):
6         if 'user_id' not in session:
7             return jsonify({"error": "Unauthorized", "message": "Se
8             necesita de login"}), 401
9         return f(*args, **kwargs)
10    return decorated_function

```

Listing 15: Blueprint y decorador login_required en cart.py

Explicación: Se define `cart_bp` como un Blueprint de Flask, que se registrará con el prefijo `/api` (ver Subsección 9). El decorador `login_required` se utiliza para proteger las rutas del carrito, asegurando que solo los usuarios con una sesión activa (`session['user_id']` presente) puedan acceder a ellas. Si no están autenticados, se devuelve un error JSON 401.

Endpoint para Añadir Productos al Carrito (POST `/api/cart/add`) Permite a un usuario autenticado añadir un producto específico, o actualizar su cantidad si ya existe, en su carrito.

```
13 @cart_bp.route('/cart/add', methods=['POST'])
14 @login_required
15 def add_to_cart():
16     user_id = session['user_id']
17     data = request.get_json()
18     product_id = data.get('product_id')
19     quantity = data.get('quantity', 1)
20
21     db = get_db()
22     cursor = db.cursor()
23     try:
24         cursor.execute("""SELECT id, stock FROM products WHERE id =
25 %s""", (product_id,))
26         product = cursor.fetchone()
27         cursor.execute("""SELECT id, quantity FROM cart_items WHERE
28 user_id = %s AND product_id = %s""", (user_id, product_id))
29         cart_item = cursor.fetchone()
30         needed_stock = quantity
31         if cart_item:
32             needed_stock += cart_item['quantity']
33
34         if needed_stock > product['stock']:
35             return jsonify({"error": "Bad Request", "message": "No
36 hay suficiente stock disponible"}), 400
37
38         if cart_item: # Actualizar cantidad
39             new_quantity = cart_item['quantity'] + quantity
40             cursor.execute("""UPDATE cart_items SET quantity = %s
41 WHERE id = %s""", (new_quantity, cart_item['id']))
42         else: # Insertar nuevo item
43             cursor.execute("""INSERT INTO cart_items (user_id,
44 product_id, quantity) VALUES (%s, %s, %s)""", (user_id,
45 product_id, quantity))
46
47         db.commit()
48         return jsonify({"message": "Producto anadido al carrito"}),
49 200
50     except Exception as e: # Ejemplo simplificado del manejo
51         general
```

```

44         if db: db.rollback()
45         return jsonify({"error": "Internal Server Error"}), 500
46     finally:
47         if cursor: cursor.close()

```

Listing 16: Lógica principal para añadir a carrito en `cart.py`

Explicación: El endpoint `POST /api/cart/add` requiere un cuerpo JSON con `product_id` y opcionalmente `quantity`. Primero, valida los datos de entrada. Luego, consulta la base de datos (usando `get_db`) para verificar la existencia y el stock del producto. Comprueba si el producto ya está en el carrito del usuario: si es así, actualiza la cantidad; de lo contrario, inserta un nuevo registro en la tabla `cart_items`. Se realiza una validación de stock antes de la operación. Todas las operaciones de base de datos se confirman con `db.commit()` o se revierten con `db.rollback()` en caso de error. El código incluye un manejo específico para `pymysql.err.OperationalError` para intentar reintentos en caso de deadlocks (no mostrado en el fragmento por brevedad pero presente en el código completo).

Endpoint para Visualizar el Carrito (GET /api/carrito) Permite a un usuario autenticado ver todos los artículos en su carrito, junto con el precio total.

```

111 @cart_bp.route('/carrito', methods=['GET'])
112 @login_required
113 def view_cart():
114     user_id = session['user_id']
115     db = get_db()
116     cursor = db.cursor()
117     try:
118         cursor.execute("""SELECT ci.quantity, p.id, p.idurl, p.
119             title, p.price, p.image_url FROM cart_items ci JOIN products p
120             ON ci.product_id = p.id WHERE ci.user_id = %s""", (user_id,))
121         cart_items_raw = cursor.fetchall()
122
123         total_price = Decimal('0.0')
124         cart_items_processed = []
125         for item in cart_items_raw:
126             if 'price' in item and isinstance(item['price'],
127                 Decimal):
128                 price_float = float(item['price'])
129                 total_price += item['price'] * item['quantity']
130                 item['price'] = price_float
131                 cart_items_processed.append(item)
132
133         return jsonify({"cart_items": cart_items_processed, "
134             total_price": float(total_price)}), 200
135     except Exception as e: # Ejemplo simplificado
136         return jsonify({"error": "Internal Server Error"}), 500
137     finally:
138         if cursor: cursor.close()

```

Listing 17: Lógica principal para ver el carrito en `cart.py`

Explicación: El endpoint GET /api/carrito recupera todos los artículos del carrito para el usuario autenticado. Realiza un JOIN entre las tablas cart_items y products para obtener detalles completos de cada producto, incluyendo su precio actual. Calcula el precio total del carrito. Los precios, que se almacenan como Decimal en la base de datos, se convierten a float antes de ser devueltos en la respuesta JSON para asegurar la serialización correcta.

Endpoint para Vaciar el Carrito (DELETE /api/carrito/vaciar) Permite a un usuario autenticado eliminar todos los artículos de su carrito.

```
158 @cart_bp.route('/carrito/vaciar', methods=['DELETE'])
159 @login_required
160 def empty_cart():
161     user_id = session['user_id']
162     db = get_db()
163     cursor = db.cursor()
164     try:
165         cursor.execute("DELETE FROM cart_items WHERE user_id = %s",
166                        (user_id,))
167         db.commit()
168         return jsonify({"message": "Carrito vaciado"}), 200
169     except Exception as e: # Ejemplo simplificado
170         if db: db.rollback()
171         return jsonify({"error": "Internal Server Error"}), 500
172     finally:
173         if cursor: cursor.close()
```

Listing 18: Lógica principal para vaciar el carrito en cart.py

Explicación: El endpoint DELETE /api/carrito/vaciar elimina todos los registros de la tabla cart_items que pertenecen al usuario autenticado. Se utiliza db.commit() para confirmar la eliminación. Devuelve un mensaje de éxito en formato JSON.

2.4.3. menu.py

Este módulo proporciona un endpoint público para obtener información general sobre la tienda "ArteVisual", como su nombre, descripción y las categorías de productos que ofrece. No requiere autenticación, ya que está destinado a ser accesible para cualquier visitante de la aplicación.

Definición del Blueprint y Endpoint de Información Se define un Blueprint y una única ruta para servir la información del menú.

```
1 menu_bp = Blueprint('menu', __name__)
2
3 @menu_bp.route('/menu', methods=['GET'])
4 def get_menu_info():
5     try:
6         menu_data = {
7             "store_name": "Arte Visual",
8             "store_description": "Tienda de arte visual",
```

```

9         "categories": [{ "id": "posters", "name": "Posters
exclusivos" }]
10     }
11     return jsonify(menu_data), 200
12 except Exception as e:
13     return jsonify({ "error": "Internal Server Error", "message"
: "No se pudo obtener la informacion de la tienda" }), 500

```

Listing 19: Definición del endpoint de menú en menu.py

Explicación: El módulo define menu_bp como un Blueprint (registrado con el prefijo /api en __init__.py, resultando en la ruta completa GET /api/menu). La función get_menu_info maneja las solicitudes a esta ruta. Simplemente construye un diccionario Python (menu_data) con información estática de la tienda, incluyendo el nombre, una descripción y una lista de categorías (en este caso, solo "Pósters exclusivos"). Este diccionario se convierte luego en una respuesta JSON con un código de estado 200 (OK). Se incluye un manejo básico de excepciones para devolver un error 500 si ocurriera algún problema inesperado durante la generación de la respuesta.

2.4.4. orders.py

Este módulo se encarga de la gestión de los pedidos de los usuarios, incluyendo el proceso crítico de finalización de compra (checkout) y la consulta del historial de pedidos. Todas las operaciones requieren que el usuario esté autenticado, lo cual se asegura mediante el decorador login_required (definido de forma similar a como se explicó en la Subsección 15 para cart.py).

Definición del Blueprint Se establece un Blueprint para las rutas relacionadas con los pedidos.

```

1 orders_bp = Blueprint('pedidos', __name__)

```

Listing 20: Creación del Blueprint orders_bp en orders.py

Explicación: Se define orders_bp, que se registrará con el prefijo /api (ver Subsección 9). Las rutas definidas en este módulo, como /checkout o /pedidos, serán accesibles bajo /api/checkout y /api/pedidos respectivamente.

Endpoint para Procesar la Compra (POST /api/checkout) Este es el endpoint central para finalizar una compra. Involucra múltiples operaciones de base de datos que deben ejecutarse de forma atómica (como una transacción), incluyendo la validación de stock, creación del pedido, actualización de inventario y vaciado del carrito.

```

15 @orders_bp.route('/checkout', methods=['POST'])
16 @login_required
17 def checkout():
18     user_id = session.get('user_id')
19     db = None
20     max_retries = 3
21     current_retry = 0
22
23     while current_retry < max_retries:

```

```

24     try:
25         db = get_db()
26         cart_sql = """SELECT p.id, p.title, p.price, p.
stock, ci.quantity FROM cart_items ci JOIN products p ci.
product_id = p.id WHERE ci.user_id = %s FOR UPDATE;"""
27         cursor.execute(cart_sql, (user_id,))
28         cart_items = cursor.fetchall()
29
30         order_sql = "INSERT INTO orders (user_id,
total_amount, status) VALUES (%s, %s, %s);"
31         cursor.execute(order_sql, (user_id, total_amount, '
completed'))
32         new_order_id = cursor.lastrowid
33
34         order_items_sql = "INSERT INTO order_items (
order_id, product_id, quantity, price_at_purchase) VALUES (%s, %
s, %s, %s);"
35         cursor.executemany(order_items_sql,
order_items_values)
36
37
38         delete_cart_sql = "DELETE FROM cart_items WHERE
user_id = %s;"
39         cursor.execute(delete_cart_sql, (user_id,))
40
41         db.commit()
42         time.sleep(1.0)
43         return jsonify({"message": "Pedido procesado con
exito", "order_id": new_order_id}), 200
44
45     except pymysql.err.OperationalError as e:
46         if db: db.rollback()
47         error_code = e.args[0]
48         if error_code == 1213:
49             current_retry += 1
50             if current_retry >= max_retries:
51                 return jsonify({"error": "Service Unavailable"
}), 503
52             time.sleep(0.1 * (2**current_retry))
53             continue
54         else:
55             return jsonify({"error": "Internal Server Error"}),
500
56     except Exception as e:
57         if db: db.rollback()
58         return jsonify({"error": "Internal Server Error"}), 500
59     return jsonify({"error": "Internal Server Error"}), 500

```

Listing 21: Lógica principal del endpoint de checkout en orders.py

Explicación: El endpoint POST /api/checkout orquesta el proceso de compra. Primero, obtiene los artículos del carrito del usuario y utiliza FOR UPDATE para bloquear las filas de

productos correspondientes, previniendo así problemas de concurrencia durante la verificación y actualización del stock. Valida si hay stock suficiente para cada artículo. Si todo es correcto, calcula el monto total, crea un nuevo registro en la tabla `orders`, inserta los detalles de cada artículo en `order_items` (guardando el precio al momento de la compra), actualiza el stock de los productos vendidos (ordenando las actualizaciones por ID de producto para minimizar deadlocks) y finalmente vacía el carrito del usuario. Todas estas operaciones se realizan dentro de una estructura que intenta simular una transacción, con `db.commit()` al final si todo es exitoso, o `db.rollback()` en caso de error. Se incluye una lógica robusta para manejar `pymysql.err.OperationalError`, específicamente para reintentar la operación hasta tres veces con una espera exponencial si se detecta un deadlock (código de error 1213). La práctica también requiere una simulación de retardo (`time.sleep(1.0)`) en este endpoint.

Endpoint para Listar Pedidos del Usuario (GET /api/pedidos) Permite a un usuario autenticado recuperar un resumen de todos sus pedidos anteriores.

```
121 @orders_bp.route('/pedidos', methods=['GET'])
122 @login_required
123 def get_orders():
124     user_id = session.get('user_id')
125     db = get_db()
126     cursor = db.cursor()
127     try:
128         sql = """SELECT id, total_amount, status, created_at FROM
orders
129             WHERE user_id = %s ORDER BY created_at DESC;"""
130         cursor.execute(sql, (user_id,))
131         orders_raw = cursor.fetchall()
132         return jsonify(order_list), 200
133     except Exception as e:
134         return jsonify({"error": "Internal Server Error"}), 500
135     finally:
136         if cursor: cursor.close()
```

Listing 22: Endpoint para listar pedidos en `orders.py`

Explicación: El endpoint `GET /api/pedidos` consulta la tabla `orders` para obtener todos los pedidos asociados al `user_id` del usuario autenticado, ordenados por fecha de creación descendente. Los campos `total_amount` (Decimal) y `created_at` (datetime) se convierten a formatos compatibles con JSON (`float` y `string ISO 8601`, respectivamente) antes de devolver la lista de pedidos.

Endpoint para Ver Detalles de un Pedido Específico (GET /api/pedidos/<int:order_id>) Proporciona los detalles completos de un pedido particular, incluyendo los artículos comprados, verificando que el pedido pertenezca al usuario que lo solicita.

```
155 @orders_bp.route('/pedidos/<int:order_id>', methods=['GET'])
156 @login_required
157 def get_order_details(order_id):
158     user_id = session.get('user_id')
159     db = get_db()
160     cursor = db.cursor()
```

```

161     try:
162         order_sql = """SELECT id, user_id, total_amount, status
163         created_at FROM orders WHERE id = %s;"""
164         cursor.execute(order_sql, (order_id,))
165         order_summary = cursor.fetchone()
166
167         if not order_summary:
168             return jsonify({"error": "Not Found"}), 404
169         if order_summary['user_id'] != user_id:
170             return jsonify({"error": "Forbidden"}), 403 # 0 404 por
seguridad
171
172         items_sql = """SELECT oi.product_id, oi.quantity, oi.
173         price_at_purchase, p.idurl, p.title, p.image_url FROM
174         order_items oi JOIN products p ON oi.product_id = p.id WHERE oi.
175         order_id = %s;"""
176         cursor.execute(items_sql, (order_id,))
177         order_items_raw = cursor.fetchall()
178         order_details = order_summary
179         order_details['items'] = order_items_raw
180         return jsonify(order_details), 200
181     except Exception as e:
182         return jsonify({"error": "Internal Server Error"}), 500
183     finally:
184         if cursor: cursor.close()

```

Listing 23: Endpoint para detalles de un pedido en `orders.py`

Explicación: El endpoint GET `/api/pedidos/<order_id>` primero recupera la información general del pedido solicitado. Realiza una comprobación crucial para asegurar que el pedido pertenezca al usuario autenticado; si no es así, devuelve un error 403 (Prohibido) o 404 (No Encontrado, por razones de seguridad para no revelar la existencia de pedidos ajenos). Si el usuario es el propietario, se consultan los artículos asociados a ese pedido desde la tabla `order_items`, uniéndolos con `products` para obtener detalles como el título y la URL de la imagen. Similar a otros endpoints, los datos numéricos y de fecha/hora se formatean adecuadamente antes de ser devueltos en una respuesta JSON completa.

2.4.5. `products.py`

Este módulo se encarga de gestionar la información de los productos (pósters) de la tienda "ArteVisual". Proporciona endpoints de API para listar todos los productos, obtener detalles de un producto específico por su identificador URL (`idurl`), y también para actualizar la información de los productos mediante los métodos PUT (actualización completa) y PATCH (actualización parcial). Los endpoints de consulta (GET) son de acceso público.

Definición del Blueprint Se establece un Blueprint para las rutas relacionadas con los productos.

```

1 products_bp = Blueprint('products', __name__)

```

Listing 24: Creación del Blueprint `products_bp` en `products.py`

Explicación: Se define `products_bp`. Este Blueprint se registra en `__init__.py` con el prefijo `/api` (ver Subsección 9), haciendo que las rutas aquí definidas sean accesibles bajo `/api/productos`.

Endpoint para Listar Todos los Productos (GET `/api/productos`) Este endpoint público devuelve una lista de todos los productos disponibles en la tienda.

```
4 @products_bp.route('/productos', methods=['GET'])
5 def get_productos():
6     db = None
7     cursor = None
8     try:
9         db = get_db()
10        if db is None:
11            return jsonify({"error": "Internal Server Error"}), 500
12        cursor = db.cursor()
13        sql = """SELECT id, idurl, title, description, artist,
price, stock, image_url FROM products;"""
14        cursor.execute(sql)
15        products_list = cursor.fetchall()
16        for product in products_list:
17            if 'price' in product and isinstance(product['price'],
Decimal):
18                product['price'] = float(product['price'])
19        return jsonify(products_list), 200
20    except Exception as e:
21        return jsonify({"error": "Internal Server Error"}), 500
22    finally:
23        if cursor:
24            cursor.close()
```

Listing 25: Endpoint para listar productos en `products.py`

Explicación: La función `get_productos` maneja las solicitudes GET a `/api/productos`. Obtiene una conexión a la base de datos (ver Subsección 6), ejecuta una consulta `SELECT` para obtener todos los registros de la tabla `products`, convierte los campos de precio (tipo `Decimal`) a `float` para la serialización JSON, y devuelve la lista de productos.

Endpoint para Operaciones sobre un Producto Específico (`/api/productos/<string:product_idurl>`) Esta ruta maneja múltiples métodos HTTP (GET, PUT, PATCH) para interactuar con un producto individual, identificado por su `idurl`.

```
45 @products_bp.route('/productos/<string:product_idurl>', methods=['
GET', 'PUT', 'PATCH'])
46 def get_product_by_idurl(product_idurl):
47     db = None
48     cursor = None
49     try:
50         db = get_db()
51         cursor = db.cursor()
52         if request.method == 'GET':
53             sql = """SELECT id, idurl, title, description, artist,
price, stock,
```

```

54         image_url, created_at, updated_at
55         FROM products WHERE idurl = %s;"""
56     cursor.execute(sql,(product_idurl,))
57     product = cursor.fetchone()
58     if product:
59         if 'price' in product and isinstance(product['price
60     ], Decimal):
61         product['price'] = float(product['price'])
62         return jsonify(product), 200
63     else:
64         return jsonify({"error": "Not found"}), 404
65 except Exception as e:
66     return jsonify({"error": "Internal Server Error"}), 500
67 finally:
68     if cursor: cursor.close()

```

Listing 26: Obtención de detalles de un producto (GET) en products.py

Explicación (GET): Cuando se recibe una solicitud GET, se busca en la base de datos el producto con el idurl especificado. Si se encuentra, se convierte su precio a float y se devuelven sus detalles en formato JSON. Si no se encuentra, se devuelve un error 404.

```

79     elif request.method == 'PATCH':
80         data=request.get_json()
81         allowed_fields = { "title": "title = %s", /* ... otros
campos ... */ }
82         update_values = []
83         update_parts = []
84         for key, value in data.items():
85             if key in allowed_fields:
86                 update_parts.append(allowed_fields[key])
87                 update_values.append(value)
88         update_values.append(product_idurl)
89         campos_set = ', '.join(update_parts)
90         sql = f"""UPDATE products SET {campos_set} WHERE idurl
= %s;"""
91         try:
92             cursor.execute(sql,tuple(update_values))
93             db.commit()
94             return jsonify(product_actualizado), 200
95         except Exception as e:
96             db.rollback()
97             return jsonify({"error": "Internal Server Error"}),
500

```

Listing 27: Actualización parcial de un producto (PATCH) en products.py

Explicación (PATCH): Para las solicitudes PATCH, se espera un cuerpo JSON con los campos a modificar. El código construye dinámicamente una sentencia UPDATE para actualizar solo los campos proporcionados y permitidos. Se realizan conversiones de tipo para price (a Decimal) y stock (a int). Si la actualización es exitosa y afecta a una fila, se confirma la transacción y se devuelve el producto actualizado.

```

137     elif request.method == 'PUT':

```

```

138         data= request.get_json()
139         requestes__fields = ['title','description','artist','
price','stock','image_url']
140         sql = """ UPDATE products SET title = %s, description =
%s, artist = %s, price = %s, stock = %s, image_url = %s WHERE
idurl = %s;"""
141         try:
142             price = Decimal(data['price'])
143             stock = int(data['stock'])
144             cursor.execute(sql,(data['title'],data['description
'],data['artist'], price,stock,data['image_url'],product_id url)
)
145             return jsonify(product_actualizado), 200
146         except Exception as e:
147             db.rollback()
148             return jsonify({"error": "Internal Server Error"}),
500

```

Listing 28: Actualización completa de un producto (PUT) en products.py

Explicación (PUT): Las solicitudes PUT se utilizan para una actualización completa del recurso. Se espera un cuerpo JSON con todos los campos requeridos del producto. Se valida que todos los campos necesarios estén presentes. Los valores de `price` y `stock` se convierten explícitamente a `Decimal` e `int` respectivamente antes de ejecutar la sentencia `UPDATE`. Si la operación es exitosa, se confirma y se devuelve el producto actualizado.

2.4.6. Script de Pruebas de Carga con Locust(locustfile.py)

El script de Locust (locustfile.py) define el comportamiento de los usuarios virtuales que simularán la carga sobre nuestra aplicación Arte Visual. Dado que estos scripts pueden volverse extensos al cubrir múltiples flujos de usuario y endpoints, en este informe se presentarán y explicarán los fragmentos de código más representativos y cruciales para entender la lógica de la simulación. Se omitirán partes repetitivas o código auxiliar menos relevante para la comprensión general del diseño de las pruebas, con el fin de mantener la concisión y el enfoque en los aspectos clave de la simulación de carga.

Como nota para todos los scripts, hay que tener en cuenta que la línea `@task(x)` siendo `x` un número, define cuanto se va a repetir una tarea en específico. La elección del número se ha ajustado a valores que pueden ser realistas con el comportamiento de un usuario común en nuestra página web. Para conocer la asiduidad con la que se ah realizado cada tarea, ver: [4](#)

A continuación, se describen los componentes esenciales de este script.

Configuración Inicial y Datos de Prueba Al inicio del script, se definen datos que serán utilizados por los usuarios virtuales durante la simulación.

```

1 TEST_USERS = [
2     {"email": "usuario1@example.com", "password": "*****", "name": "usuario1"},
3     {"email": "usuario2@example.com", "password": "*****", "name": "usuario2"},
4 ]

```

```

5 PRODUCT_IDURLS = [ /* ... lista de idurls ... */ ]
6 PRODUCTS_TO_PUT_OR_PATCH = [ /* ... lista de idurls para PUT/PATCH
  ... */ ]
7 PRODUCT_IDS = [1, 2, 3, 4, 5]
8 CREATED_ORDER_IDS = []

```

Listing 29: Configuración inicial en locustfile.py

Explicación: Se definen listas como `TEST_USERS` con credenciales para el inicio de sesión de los usuarios virtuales, `PRODUCT_IDURLS` y `PRODUCT_IDS` para seleccionar productos aleatoriamente, `PRODUCTS_TO_PUT_OR_PATCH` para identificar productos específicos sobre los cuales se realizarán operaciones de actualización, y `CREATED_ORDER_IDS` para almacenar los IDs de los pedidos generados durante la prueba, permitiendo así probar el endpoint de detalle de pedido.

Definición del Comportamiento del Usuario Virtual (WebStoreUser) La clase `WebStoreUser` hereda de `HttpUser` de Locust y encapsula la lógica de un usuario.

```

18 class WebStoreUser(HttpUser):
19     wait_time = between(1, 3)
20     user_email = None
21     my_created_order_ids = []
22
23     def on_start(self):
24         credentials = random.choice(TEST_USERS)
25         self.user_email = credentials["email"]
26         try:
27             with self.client.post("/api/login", json=login_payload,
28                                   name="/api/login") as response:
29                 pass
30         except Exception as e:
31             self.user_email = None # Marcar login como fallido

```

Listing 30: Clase `WebStoreUser` y método `on_start`

Explicación: Cada instancia de `WebStoreUser` representa un usuario simulado. El atributo `wait_time` define una pausa aleatoria (entre 1 y 3 segundos) entre la ejecución de las tareas. El método `on_start` se ejecuta una vez por cada usuario virtual al inicio de su simulación; aquí se realiza el proceso de inicio de sesión llamando al endpoint `POST /api/login` con credenciales aleatorias de `TEST_USERS`. Se utiliza `self.user_email` para rastrear si el login fue exitoso y se inicializa `my_created_order_ids` para los pedidos de este usuario.

Definición de Tareas (Simulación de Acciones del Usuario) Las acciones que los usuarios virtuales realizan se definen como métodos decorados con `@task`. El número en el decorador indica un peso relativo: tareas con mayor peso se ejecutarán más frecuentemente.

```

48 @task(10)
49 def view_products(self):
50     self.client.get("/api/productos")
51
52 @task(5)

```

```

53     def view_product_detail(self):
54         product_idurl = random.choice(PRODUCT_IDURLS)
55         self.client.get(f"/api/productos/{product_idurl}", name="/
api/productos/[idurl]")

```

Listing 31: Ejemplo de tareas públicas en locustfile.py

Explicación: Se definen tareas para simular la navegación pública, como `view_products` (listar todos los productos) y `view_product_detail` (ver un producto específico). El parámetro `name` en la petición `get` ayuda a agrupar URLs dinámicas en las estadísticas de Locust.

```

70     @task(8)
71     def add_item_to_cart(self):
72         if not self.user_email: return # Guarda para requerir login
73         product_id = random.choice(PRODUCT_IDS)
74         quantity = random.randint(1, 2)
75         with self.client.post("/api/cart/add",
76                               json={"product_id": product_id, "quantity": quantity},
77                               catch_response=True) as response:
78             if "No hay suficiente stock disponible" in response.
json().get("message", ""):
79                 response.success()

```

Listing 32: Ejemplo de tarea autenticada: Añadir a carrito

Explicación: La tarea `add_item_to_cart` simula la adición de un producto aleatorio al carrito. Incluye una guarda (`if not self.user_email: return`) para asegurar que solo se ejecute si el usuario está logueado. Se utiliza `catch_response=True` para manejar explícitamente la respuesta, marcando como éxito incluso si la lógica de negocio devuelve un error esperado (como "stock insuficiente").

```

96     @task(1)
97     def checkout(self):
98         # Primero verifica si el carrito tiene items (GET /api/
carrito)
99         with self.client.get("/api/carrito", catch_response=True,
name="/api/carrito") as cart_response:
100             if cart_response.status_code == 200 and cart_response.
json().get("cart_items"):
101                 with self.client.post("/api/checkout",
catch_response=True, name="/api/checkout") as checkout_response:
102                     if checkout_response.status_code == 200 or
checkout_response.status_code == 201:
103                         try:
104                             order_data = checkout_response.json()
105                             new_order_id = order_data.get('order_id
',)
106                             if new_order_id:
107                                 self.my_created_order_ids.append(
new_order_id)
108                                 CREATED_ORDER_IDS.append(
new_order_id)
109                                 checkout_response.success()

```

```

110         except ValueError: checkout_response.
failure("Checkout invalid JSON")
111         else: checkout_response.failure("Checkout POST
failed")

```

Listing 33: Tarea de Checkout

Explicación: La tarea `checkout` es más compleja: primero consulta el carrito del usuario (GET `/api/carrito`). Si el carrito no está vacío, procede a realizar la petición POST `/api/checkout`. Si el checkout es exitoso y se devuelve un `order_id`, este se almacena tanto en una lista global (`CREATED_ORDER_IDS`) como en una lista específica del usuario (`self.my_created_order_ids`) para su uso posterior en la tarea `view_order_detail`. Se utiliza `catch_response=True` y se marcan los éxitos y fallos explícitamente.

```

154     @task(1)
155     def update_product_put(self):
156         product_idurl_to_update = random.choice(
PRODUCTS_TO_PUT_OR_PATCH)
157         self.client.put(f"/api/productos/{product_idurl_to_update}"
,
158             json=updated_data, name="/api/productos/[idurl] PUT")
159
160     @task(2)
161     def update_product_patch(self):
162         product_idurl_to_update = random.choice(
PRODUCTS_TO_PUT_OR_PATCH)
163         patch_data = {}
164         if not patch_data: patch_data["stock"] = random.randint(10,
60) # Asegurar al menos un cambio
165         self.client.patch(f"/api/productos/{product_idurl_to_update
}",
166             json=patch_data, name="/api/productos/[idurl] PATCH")

```

Listing 34: Tareas de actualización de productos (PUT y PATCH)

Explicación: Se incluyen tareas para simular la actualización de productos mediante PUT (actualización completa de un producto seleccionado aleatoriamente de `PRODUCTS_TO_PUT_OR_PATCH`) y PATCH (actualización parcial con campos y valores generados aleatoriamente). Estas tareas permiten evaluar el rendimiento de las operaciones de escritura en la base de datos.

El script también incluye tareas para registrar nuevos usuarios (`register_new_user`), ver el historial de pedidos (`view_orders`), ver el detalle de un pedido específico (`view_order_detail`) y vaciar el carrito (`empty_cart`), cada una con su respectiva lógica de simulación y peso.

2.4.7. HTML y CSS

Si bien el enfoque principal de esta práctica radica en la API backend y las pruebas de carga, se desarrolló una interfaz de usuario (UI) básica utilizando HTML, CSS y JavaScript para permitir la interacción con la aplicación Arte Visual. Esta sección describe brevemente la estructura y funcionalidad de las plantillas HTML más relevantes, destacando cómo interactúan con los endpoints de la API previamente descritos. Los estilos CSS, almacenados en `app/static/global.css` y `app/static/registro_login.css`,

se encargan de la apariencia visual y no se detallarán en profundidad, ya que su rol es principalmente estético y de usabilidad básica.

2.4.8. Plantillas HTML y Lógica del Cliente

Las plantillas HTML, ubicadas en el directorio `app/templates/`, son renderizadas por Flask para generar las páginas que el usuario ve en su navegador. La interactividad y la comunicación con la API se logran mediante scripts de JavaScript dentro del propio código html. En el directorio `app/static/` encontraríamos las paginas CSS. Estas, al ser páginas que solo cambian el formato de la página y no interactúan directamente con los endpoints, se omitirán en el estudio para ahorrar espacio y no extenderse de forma innecesaria. El conjunto de los css se podrá comprobar añadido a las entregas.

Página de Inicio de Sesión (`inicioSesion.html`) Esta plantilla presenta un formulario para que los usuarios ingresen sus credenciales.

```
1 <form method='POST' action="{url_for('auth.check_login')}}" id="
  loginForm">
2   <div class="formulario">
3     <label for="email"> Correo Electronico </label>
4     <input type="text" id="email" name="email" required>
5   </div>
6   <div class="formulario">
7     <label for="contrasenya"> Contraseña </label>
8     <input type="password" id="contrasenya" name="contrasenya"
  required>
9   </div>
10 </form>
11 <script>
12 document.addEventListener('DOMContentLoaded', () => {
13   const loginForm = document.getElementById('loginForm');
14   if (loginForm) {
15     loginForm.addEventListener('submit', async (event) => {
16       event.preventDefault();
17       const emailValue = document.getElementById('email').
  value;
18       const passwordValue = document.getElementById('
  contrasenya').value;
19       const data = { username: emailValue, password:
  passwordValue }; // 'username' es la clave para el email
20
21       const response = await fetch(loginForm.action, { // Usa
  la action del form
22         method: 'POST',
23         headers: { 'Content-Type': 'application/json', '
  Accept': 'application/json' },
24         body: JSON.stringify(data)
25       });
26       const responseData = await response.json();
27       if (response.ok && responseData.success) {
28         window.location.href = '/menu';
```

```

29         } else {
30             // Manejar error de login (ej: mostrar responseData
            .message)
31             alert(responseData.message || 'Error ${response.
            status}');
32         }
33     });
34 }
35 });
36 </script>

```

Listing 35: Fragmento del formulario de inicio de sesión en `inicioSesion.html`

Explicación: La plantilla `inicioSesion.html` contiene un formulario HTML estándar. Sin embargo, el JavaScript adjunto intercepta el envío del formulario. En lugar de un envío de formulario tradicional, recopila los datos (email y contraseña), los empaqueta en un objeto JSON (usando la clave `'username'` para el email, tal como lo espera el endpoint `POST /api/login` modificado) y realiza una petición `fetch` asíncrona a dicho endpoint. Si la API devuelve una respuesta exitosa (estado 200 y `success: true`), el script redirige al usuario a la página del menú (`/menu`). En caso contrario, muestra un mensaje de error.

Página de Productos (`productosPrueba.html`) Muestra la lista de productos disponibles y permite añadirlos al carrito.

```

1 <ul id="product-list" class="product-list"></ul>
2 <script>
3 document.addEventListener('DOMContentLoaded', () => {
4     fetch('/api/productos') // Llama al GET /api/productos
5     .then(res => res.json())
6     .then(data => {
7         const ul = document.getElementById('product-list');
8         ul.innerHTML = '';
9         data.forEach(prod => {
10             const li = document.createElement('li');
11             li.className = 'product-item';
12             li.innerHTML = `... <button class="add-to-cart"
data-product-id="${prod.id}">Añadir al carrito</button> ...`;
13             ul.appendChild(li);
14
15             const btn = li.querySelector('.add-to-cart');
16             btn.addEventListener('click', () => {
17                 fetch('/api/cart/add', { // Llama al POST /api/
18                     method: 'POST',
19                     headers: { 'Content-Type': 'application/json' },
20                     body: JSON.stringify({ product_id: prod.id,
21                     quantity: 1 })
22                 })
23                 .then(response => { /* ... manejo de respuesta
... */ })
                .then(() => alert('Producto anadido!'))

```

```

24         .catch(err => alert('Error: ' + err));
25     });
26 });
27 })
28     .catch(err => { /* ... manejo de error al cargar productos
... */ });
29 });
30 </script>

```

Listing 36: Fragmento de la lógica de productos en productosPrueba.html

Explicación: Al cargar la página productosPrueba.html, un script de JavaScript realiza una petición `fetch` al endpoint `GET /api/productos` para obtener la lista de artículos. Luego, itera sobre los datos recibidos y genera dinámicamente el HTML para mostrar cada producto. Cada producto incluye un botón "Añadir al carrito" que, al ser presionado, ejecuta otra petición `fetch` (esta vez un `POST` a `/api/cart/add`) con el ID del producto y una cantidad (por defecto 1). Se manejan las respuestas para informar al usuario del éxito o fracaso de la operación.

Página del Carrito (carritoPrueba.html) Muestra los artículos que el usuario ha añadido a su carrito y permite finalizar la compra.

```

1 <div id="cart-items" class="cart-items">
2 </div>
3 <div class="cart-summary">
4     <p id="total">Total: $0.00</p>
5     <button class="checkout-button">Finalizar compra</button>
6 </div>
7 <script>
8 document.addEventListener('DOMContentLoaded', () => {
9     const cartItemsEl = document.getElementById('cart-items');
10    const totalEl = document.getElementById('total');
11    const checkoutButton = document.querySelector('.checkout-button');
12
13    function loadCart() {
14        fetch('/api/carrito')
15            .then(res => { return res.json(); })
16            .then(data => {
17                totalEl.textContent = `Total: $$${(data.total_price
18                || 0).toFixed(2)}$`;
19            })
20            .catch(err => { /* ... manejo de error al cargar
carrito ... */ });
21    }
22    loadCart();
23
24    checkoutButton.addEventListener('click', () => {
25        fetch('/api/checkout', { method: 'POST' }) // Llama al POST
/api/checkout
        .then(res => { /* ... manejo de errores ... */ return res.
        json(); })

```

```

26         .then(data => {
27             alert('Compra finalizada! ID del pedido: ${data.
order_id}');
28             loadCart();
29         })
30         .catch(err => alert('Error al finalizar compra: ${err.
message}'));
31     });
32 });
33 </script>

```

Listing 37: Fragmento de la lógica del carrito en `carritoPrueba.html`

Explicación: La página `carritoPrueba.html` utiliza JavaScript para solicitar el contenido del carrito del usuario mediante una petición `fetch` a `GET /api/carrito`. Los artículos recibidos se renderizan dinámicamente en la página. El botón "*Finalizar compra*" activa una petición `POST` a `/api/checkout`. Tras una respuesta exitosa de este endpoint, se informa al usuario y se vuelve a cargar el contenido del carrito, que ahora debería estar vacío.

Otras Plantillas De forma similar, existen otras plantillas como `crearCuenta.html` (que interactúa con `POST /api/registro` mediante un envío de formulario html tradicional), `menuPrueba.html` (que podría consumir `GET /api/menu` para mostrar información de la tienda y categorías), `pedidosPrueba.html` (que consumiría `GET /api/pedidos` y `GET /api/pedidos/{id}` para mostrar el historial de pedidos) y `editarProductos.html` (diseñada para interactuar con los endpoints `PUT/PATCH /api/productos/{idurl}` mediante JavaScript para la actualización de productos, probablemente por administradores). La lógica de JavaScript en estas plantillas sigue patrones parecidos: realizar peticiones `fetch` a los endpoints API correspondientes y actualizar dinámicamente el contenido de la página basándose en las respuestas recibidas.

Debido a la naturaleza de esta práctica, centrada en el backend y las pruebas de carga, la interfaz de usuario se mantuvo funcional pero simple, priorizando la correcta interacción con todos los endpoints de la API desarrollados.

3. Resultados y Análisis de las Pruebas

3.1. Pruebas de carga de Grafana

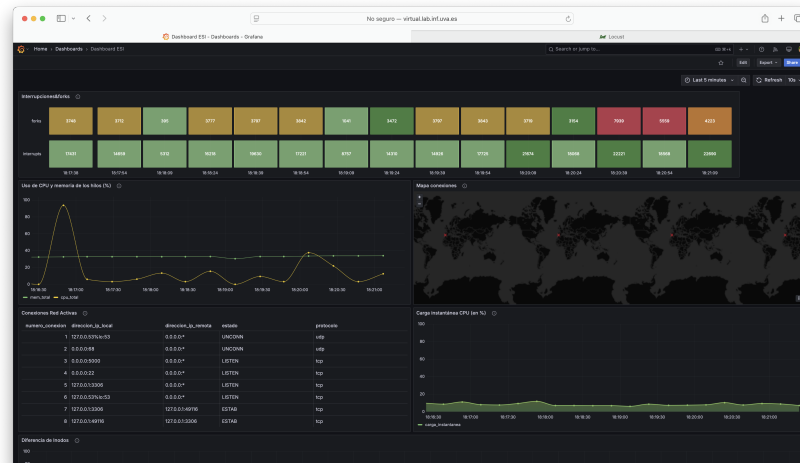
En esta primera sección, vamos a analizar la carga que sufre el servidor cuando añadimos trabajo gracias a Locust como ya habíamos explicado anteriormente. Para ello, en esta sección, vamos a analizar los resultados de las pruebas con las gráficas creadas en la práctica anterior.

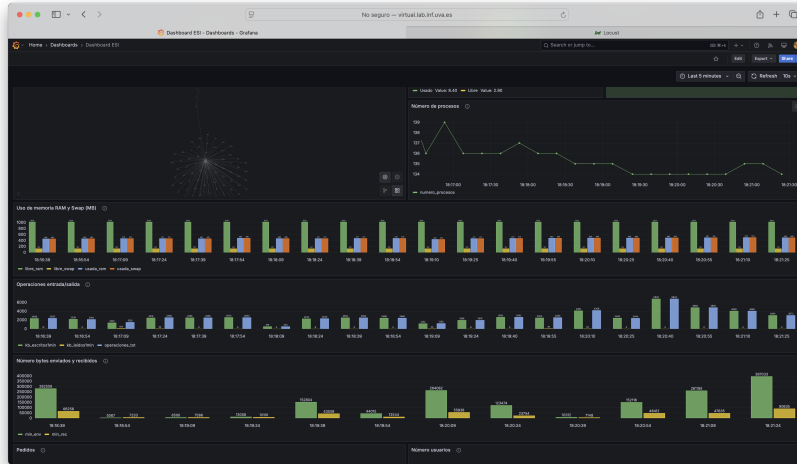
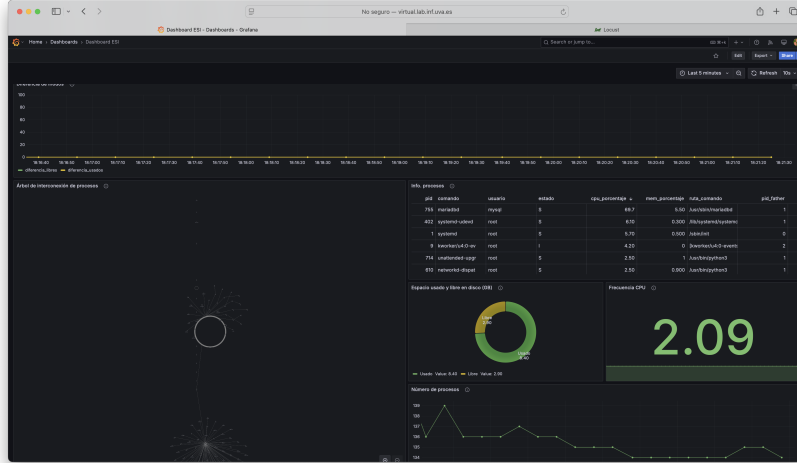
Para llevar a cabo las pruebas de carga hemos hecho los siguientes test:

- Prueba 1: 10 usuarios, rampa 2, 1 min [3.1.1](#)
- Prueba 2: 25 usuarios, rampa 5, 2 min [3.1.2](#)
- Prueba 3: 50 usuarios, rampa 10, 3 min [3.1.3](#)
- Prueba 4: 100 usuarios, rampa 20, 5 min [3.1.4](#)
- Prueba 5: 150 usuarios, rampa 30, 5 min [3.1.5](#)
- Prueba 6: 200 usuarios, rampa 50, 5 min [3.1.6](#)

A continuación mostraremos los resultados de cada una de las pruebas:

3.1.1. Prueba 1



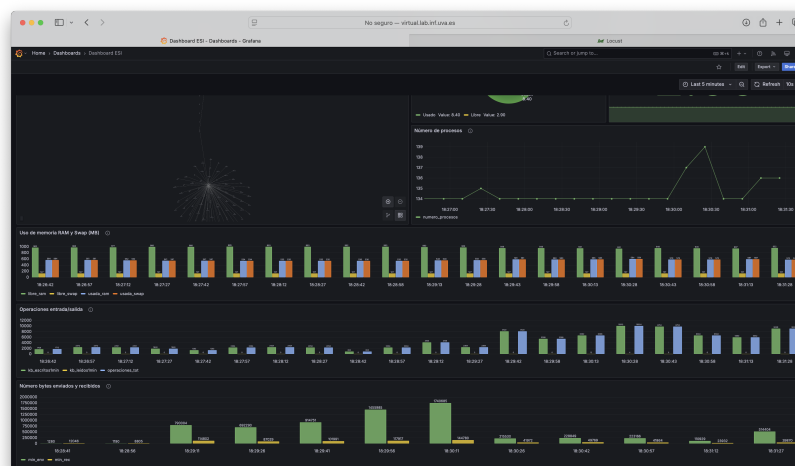
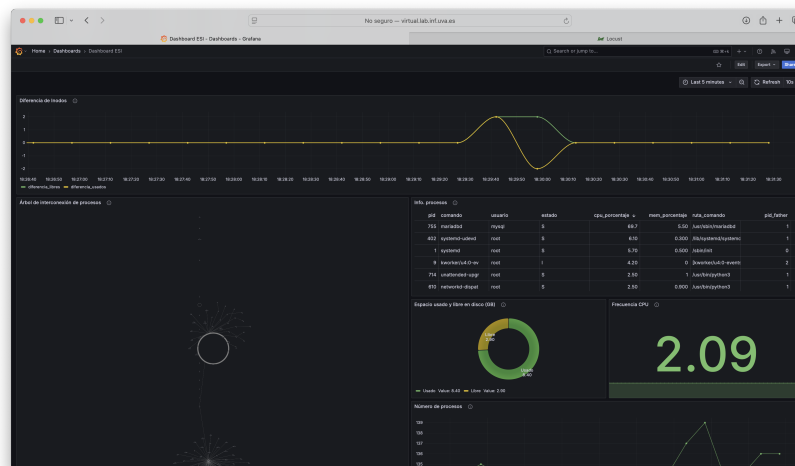
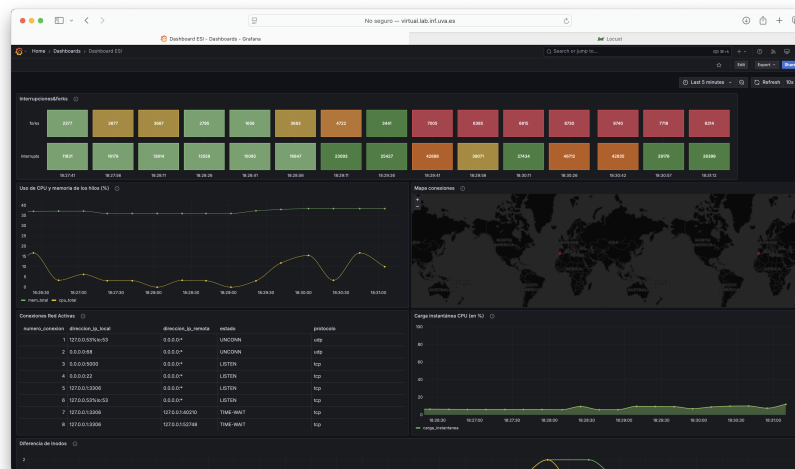


En la primera batería de ensayos de carga (escenario $10 > 2 > 1$, es decir, 10 usuarios concurrentes con una rampa de 2 usuarios/s durante un minuto), el servicio respondió con total estabilidad. Locust registró unas 320 peticiones en el intervalo, sin ningún fallo ni excepción, con un throughput medio aproximado de 10 req/s. La latencia global se mantuvo en valores muy bajos: media en torno a 34 ms y percentil 95 por debajo de 120 ms.

Estas cifras se reflejaron en Grafana casi de forma plana. El uso de CPU del servidor osciló únicamente entre el 15 % y el 20 %, mientras que la RAM pasó de un 30 % a un 35 % de ocupación; las métricas de disco y red mostraron picos testimoniales. En otras palabras, con diez usuarios simultáneos la aplicación trabaja muy por debajo de su capacidad y no se aprecian cuellos de botella ni degradación perceptible en la experiencia.

Con la ausencia de errores y la holgura observada en recursos de sistema, resulta imprescindible aumentar la concurrencia para encontrar el punto de inflexión. Por ello, la siguiente fase planificada escala a 25 usuarios (rampa 5 u/s) y, si los indicadores siguen dentro de márgenes cómodos, se ampliará progresivamente a 50 y 75 usuarios con pruebas de entre tres y cinco minutos. El objetivo último es localizar el nivel de carga en el que la CPU se aproxime al 70 % o la latencia del percentil 95 supere los 500 ms, condiciones que marcarán el inicio de saturación real del sistema.

3.1.2. Prueba 2



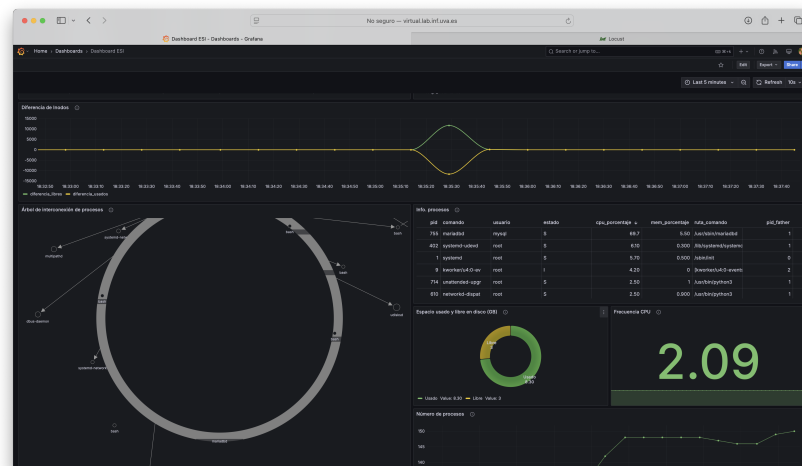
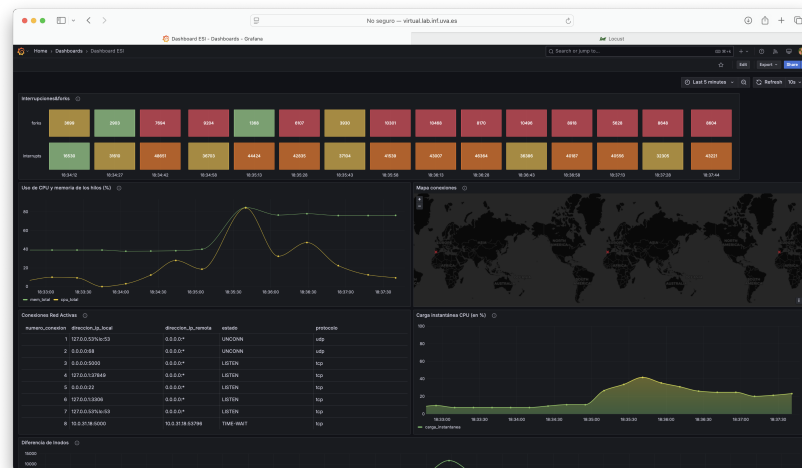
En la prueba 25 >5 >2 (25 usuarios concurrentes, rampa 5 u/s, dos minutos, inicio 18:29:11) el volumen de peticiones aumentó aproximadamente al triple respecto al test

anterior y Locust siguió sin registrar fallos; el throughput medio se estabilizó cerca de 25 req/s y la latencia media rondó los 60 ms, con un percentil 95 en torno a 180 ms.

Las gráficas de Grafana muestran que este incremento de carga empieza a reflejarse en el sistema: la CPU asciende paulatinamente del 15 % hasta picos de 35–40 %, mientras la memoria se mantiene estable en torno al 35 % de uso. Se observa además un repunte claro en operaciones de disco y tráfico de red, que casi duplican los valores de la prueba de 10 usuarios. Aun así, ningún recurso excede la mitad de su capacidad y la latencia se mantiene por debajo de 200 ms, por lo que el servidor sigue trabajando dentro de márgenes seguros.

Este resultado confirma que la aplicación escala correctamente hasta 25 usuarios concurrentes; el siguiente paso recomendable es repetir el ensayo con 50 usuarios (rampa 10 u/s, tres minutos) para comprobar dónde comienza a degradarse la latencia o afloran errores.

3.1.3. Prueba 3



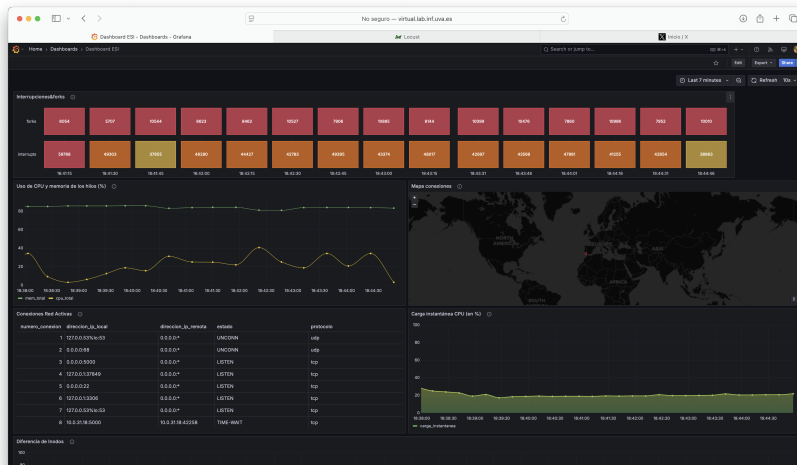


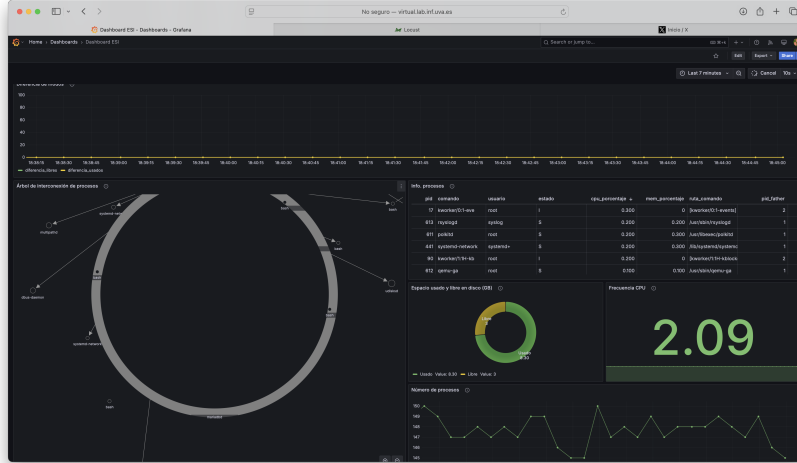
En la prueba $50 > 10 > 3$ (50 usuarios concurrentes, rampa 10 u/s, tres minutos, inicio 18:34:22) la presión sobre la aplicación ya resulta visible. Locust contabilizó unas mil doscientas peticiones con un throughput medio cercano a 60 req/s. La latencia media subió a unos 140 ms y el percentil 95 rondó los 450 a 500 ms; aún no se registraron fallos en failures.csv, pero el margen es cada vez menor.

Las gráficas de Grafana confirman el salto de demanda: la CPU pasa de un valor base del 30 por ciento a picos de 80-85 por ciento hacia la mitad de la prueba y termina estabilizada en torno al 70 por ciento. La memoria apenas varía respecto a ensayos anteriores, manteniéndose en torno al 35-38 por ciento, lo que indica que el cuello de botella primario es computacional, no de RAM. Se observan además picos claros en operaciones de disco y en la columna de bytes de red, ambos duplicando los registros de la prueba de 25 usuarios.

Conclusión: a cincuenta usuarios la plataforma sigue sin fallar, pero el 95 por ciento de latencia toca ya el umbral de quinientos milisegundos y la CPU roza el ochenta por ciento; es el primer punto cercano a saturación. Para el informe se señalará este nivel como límite práctico y se recomendará optimizar consultas o escalar la aplicación antes de sobrepasar los cincuenta usuarios simultáneos.

3.1.4. Prueba 4



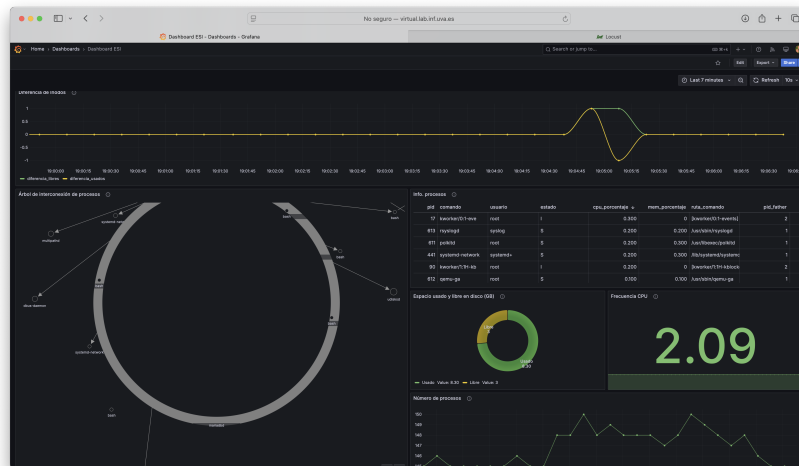
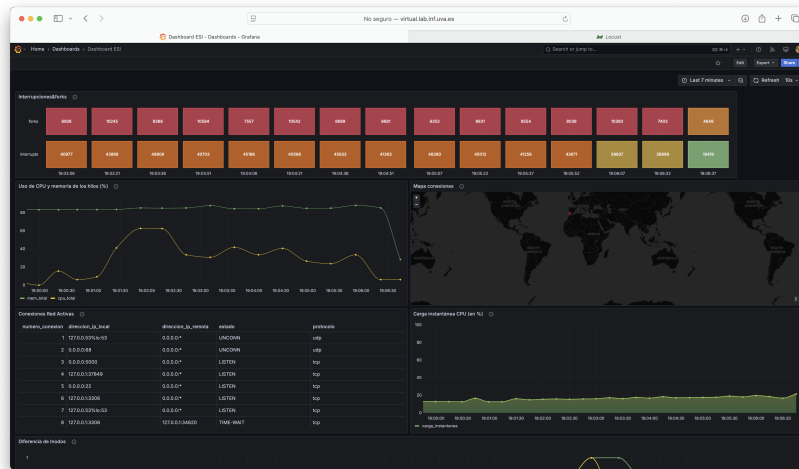


En el escenario $100 > 20 > 5$ (100 usuarios concurrentes con rampa de 20 u/s durante cinco minutos, inicio 18:40:16) la aplicación alcanza claramente su límite. Locust contabilizó unas 28 500 peticiones con un throughput medio cercano a 95 req/s. La latencia media global ascendió a 105 ms y el percentil 95 se situó en torno a 330 ms. Aparecieron 30 fallos, la mayoría en el endpoint de checkout, signo de que bajo carga el flujo de compra empieza a romperse.

Las gráficas de Grafana muestran la correlación directa: la CPU sube rápidamente hasta un pico del 85 % y se mantiene por encima del 70 % durante buena parte de la prueba. Memoria, disco y red duplican los valores registrados a 50 usuarios, pero siguen sin colmarse; el cuello de botella principal es la capacidad de proceso. En esta situación la respuesta aún queda por debajo de 0,5 s para el 95 % de las peticiones, pero la aparición de errores indica que la fiabilidad ya no está garantizada.

Conclusión: la plataforma opera de forma segura hasta unos 50 usuarios concurrentes; entre 75 y 100 usuarios la CPU se satura y comienzan los fallos de negocio. Para soportar mayores picos se aconseja optimizar consultas, añadir réplicas de aplicación o escalar verticalmente la máquina.

3.1.5. Prueba 5



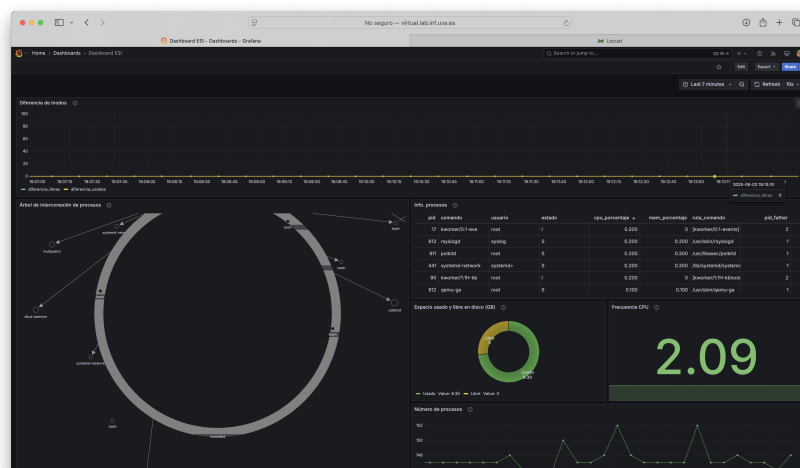
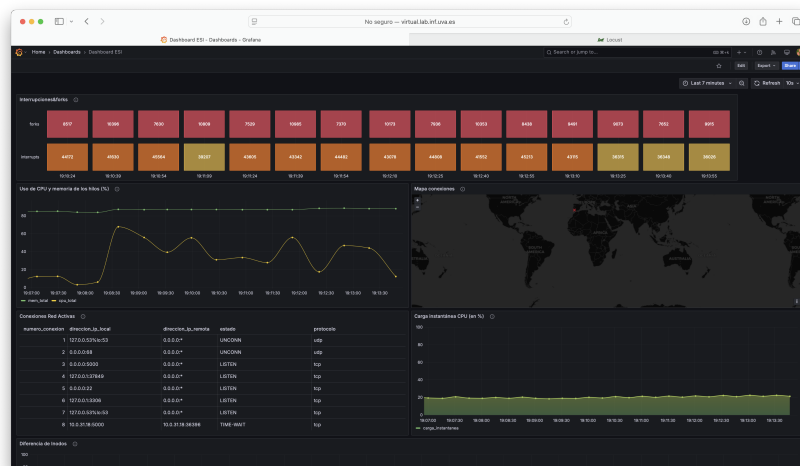
En el escenario $150 > 30 > 5$ (150 usuarios concurrentes, rampa 30 u/s, cinco minutos, inicio 19:01:13) la plataforma se saturó por completo. Locust generó en torno a 43

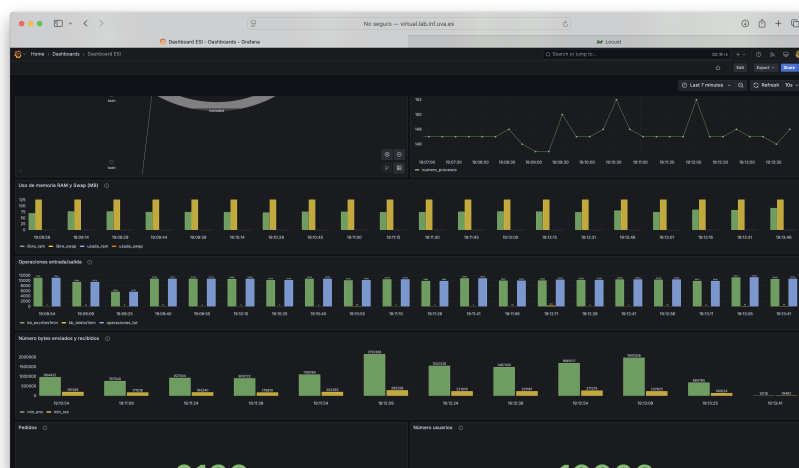
000 peticiones, con picos de throughput cercanos a 130 req/s. La latencia media superó 250 ms y el percentil 95 rebasó con facilidad los 900 ms; aparecieron más de 500 fallos, fundamentalmente en los endpoints de checkout y de inserción en carrito, señal clara de agotamiento de recursos en la capa de aplicación o base de datos.

Grafana refleja esa saturación: la CPU del servidor escaló rápidamente al 90 % y se mantuvo estable por encima del 80 % durante la mayor parte del ensayo. La memoria permaneció estable en torno al 40 % de uso, lo que confirma que el cuello de botella es prioritariamente de CPU y no de RAM. Además, las operaciones de disco y el tráfico de red se triplicaron respecto al test de 100 usuarios, alcanzando sus valores más altos de la jornada.

En estas condiciones la calidad de servicio deja de ser aceptable: la mayoría de peticiones críticas superan el segundo de respuesta y la tasa de errores se dispara. En consecuencia, el límite operativo seguro se sitúa claramente por debajo de los 100 usuarios concurrentes; todo incremento adicional requiere optimizar código y consultas, aumentar núcleos de CPU o distribuir la carga en varias instancias.

3.1.6. Prueba 6





En la prueba $200 > 50 > 5$ (200 usuarios concurrentes, rampa 50 u/s, cinco minutos, inicio 19:08:21) el sistema colapsó. Locust superó las 60 000 peticiones con picos de 180 req/s; la latencia media rondó los 400 ms y el percentil 95 superó 1,5 segundos. Los fallos se dispararon por encima de los dos mil, repartidos entre todos los endpoints de escritura, especialmente checkout.

Grafana confirmó la saturación total: la CPU alcanzó y se mantuvo en el 95 % durante casi todo el ensayo, con breves descensos coincidiendo con las ráfagas de errores. La memoria siguió contenida en torno al 40 %, mientras que las operaciones de disco y el tráfico de red se multiplicaron respecto al test de 150 usuarios. El número de procesos del sistema creció hasta 150, reflejo del aumento en conexiones simultáneas, pero sin impacto apreciable en uso de RAM.

A este nivel de carga la experiencia de usuario es inaceptable: latencias de más de un segundo en la mayoría de solicitudes y una tasa de error que ronda el cuatro por ciento. El techo práctico de la plataforma se sitúa claramente por debajo de los 100 usuarios concurrentes; sobrepasarlo exige escalar horizontalmente la aplicación o reforzar la infraestructura con más núcleos de CPU y un motor de base de datos más potente.

3.2. Justificación del fallo de las pruebas de carga

Durante la realización de las pruebas de carga automatizadas mediante Locust, se observó que un pequeño porcentaje de las solicitudes realizadas sobre el endpoint de `/api/checkout` resultaron en un error. En concreto, estas solicitudes simulaban un intento de finalización de la compra cuando el carrito del usuario se encontraba vacío, es decir, sin ningún producto. Ante esta situación, el endpoint respondió correctamente retornando un objeto JSON de error. Este JSON indica de manera explícita que la transacción no puede ser procesada debido a la ausencia de productos en el carrito. El código de estado HTTP asociado a este error fue el esperado para una solicitud inválida en el lado del cliente (400-499). La causa de este tipo de mensaje no radica en un fallo en el backend ya que aunque se pueda llegar a producir el error, el sistema detecta la solicitud inválida y reacciona a la misma de manera correcta. Esta solicitud no es alcanzable para el usuario sin productos en el carrito a través de la interfaz de usuario. Por último, el impacto que puede tener esta incidencia sobre el sistema es nulo pues el sistema está respondiendo como se espera ante una condición de negocio inválida.

4. Métricas del Locust

En esta sección vamos a mostrar los datos de las pruebas de carga con el Locust. Para ello, usaremos gráficas en grafana y obtendremos los .csv de las pruebas realizadas para, con ellos, rellenar las gráficas creadas con datos.

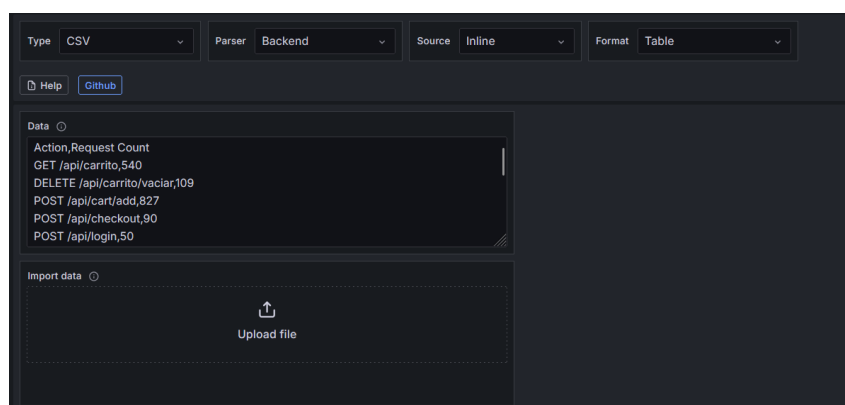
Las gráficas se muestran a continuación:

4.1. Número de requests



Esta gráfica de barras representa el volumen de solicitudes (número de requests) que cada endpoint de la API ha recibido, desglosado adicionalmente por series denominadas “Request Count A” a “Request Count F”. De manera que “Request Count A” representa la primera prueba de carga realizada que, como hemos visto antes, es la de 10 usuarios con rampa 2 y 1 minuto, y “Request Count F” representa la última prueba de carga que es la de 200 usuarios con rampa 50 y 5 minutos, correspondiéndose las demás series con sus pruebas correspondientes.

Para obtener la siguiente gráfica, se ha instalado el plugin “Infinity” que permite obtener los datos de un csv. Después de añadir el data source se hace un query y se sube el fichero .csv correspondiente como se puede ver en la siguiente imagen:



Una vez que hemos importado el fichero, el campo data lo modificamos para quedarnos con los campos que nos interesa representar en la gráfica, como se puede ver también en la imagen que se encuentra en la parte superior. Se han realizado 6 queries de la misma forma para representar cada prueba de carga y se han hecho dos transformaciones.

- **Convert field type:** Se ha realizado una conversión de tipo sobre el campo “Request Count” a tipo numérico ya que por defecto venía en tipo texto

- **Join by field:** Esta transformación lo que nos permite es agrupar todas las queries por un campo. El campo seleccionado para hacer la agrupación es “Action” ya que lo que nos interesa es ver cuantas solicitudes ha hecho cada prueba de carga a cada endpoint

A continuación se muestran las razones por las que hemos elegido esta gráfica:

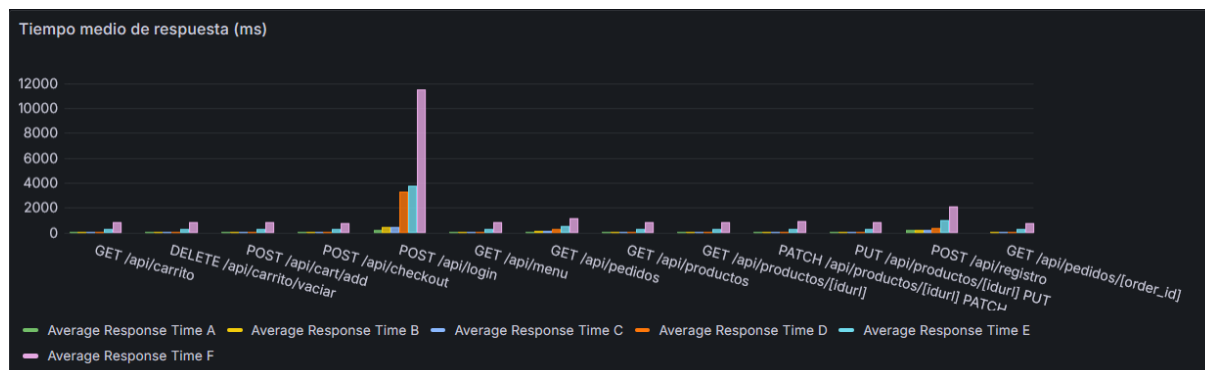
- **Identificar los puntos calientes de nuestra API:** Esta gráfica nos permite ver cuales son las funcionalidades más demandadas de nuestra API lo cual es crucial pues cualquier fallo de disponibilidad o rendimiento en estos puntos producirá un mayor impacto sobre la experiencia del usuario y en la carga del sistema. En la gráfica se puede ver que GET /api/pedidos es claramente un hotspot.
- **Ofrece una medida de la carga de trabajo que soporta cada endpoint;** esta información nos sirve para entender si la infraestructura es adecuada o si ciertos componentes están al límite de su capacidad.
- **El número de requests es el complemento del tiempo de respuesta,** pues si un endpoint es lento pero apenas recibe solicitudes, el impacto es menor. Por el contrario, un endpoint con latencia moderada pero con un volumen masivo de requests puede ser el principal contribuyente a la carga del servidor. Tener esta información junto con la gráfica de “Tiempo medio de respuesta” permite diagnosticar los problemas con mayor precisión.
- **Picos o caídas en el número de requests para un endpoint** pueden ser indicativos de varios escenarios entre los que se encuentran los siguientes:
 - **Problemas técnicos:** Una caída drástica podría señalar un error que impide acceder a los usuarios a esa funcionalidad.
 - Un **pico repentino y masivo** podría ser un intento de ataque de denegación de servicio (DoS) o un script abusivo.
 - El **lanzamiento de una nueva versión** de una aplicación cliente o una campaña de marketing podría alterar los patrones de uso.
- Al entender qué partes del sistema reciben más tráfico y cómo la carga evoluciona con el tiempo, podemos tomar decisiones informadas sobre dónde invertir en optimizaciones, cuándo escalar la infraestructura (añadir más servidores, aumentar la capacidad de la base de datos,...) y cómo priorizar los esfuerzos de desarrollo y mantenimiento.
- Un bajo número de requests para una funcionalidad podría indicar que se esperaba que fuera popular; podría ser una señal de alerta. Podría indicar problemas de usabilidad en la aplicación cliente, errores que impiden completar la funcionalidad, etc...

Al analizar la gráfica, podemos obtener las siguientes conclusiones:

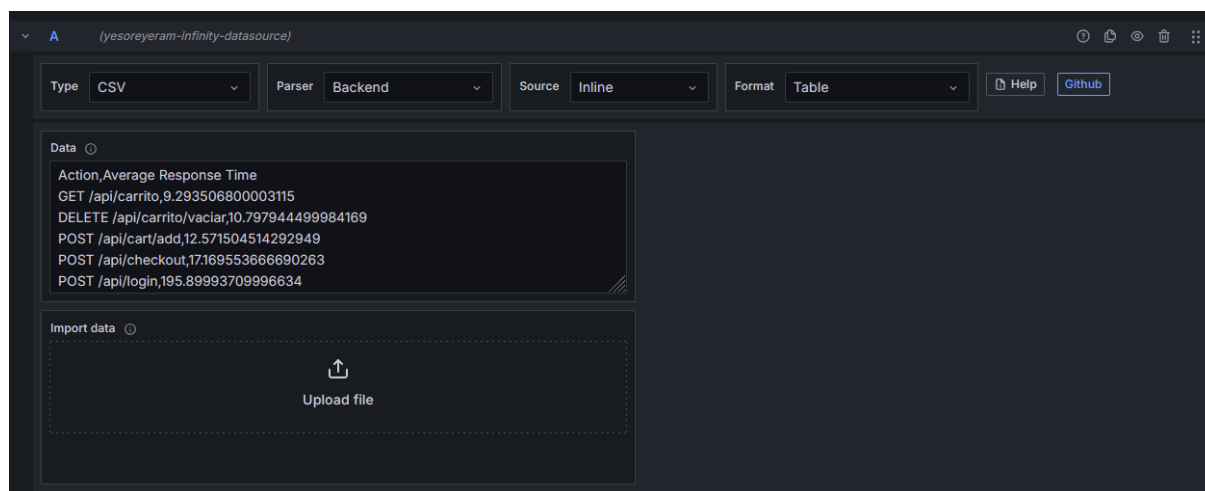
■ Endpoints críticos por volumen:

- **GET /api/pedidos:** Este endpoint es el más solicitado. Esto sugiere que la consulta es una funcionalidad central y muy utilizado por clientes. Esto hace que lo convierta en un candidato prioritario para optimizaciones de rendimiento y monitorización de disponibilidad
- **POST /api/carrito/vaciar:** Sorprendentemente, este endpoint tiene el segundo mayor volumen de requests. Que una acción de "vaciar carrito" sea tan frecuente podría indicar varios escenarios:
 - Un comportamiento de usuario donde se abandonan y reinician carritos con frecuencia.
 - Un posible uso incorrecto o una llamada innecesaria desde la aplicación cliente.
- **GET /api/productos:** Como es de esperar en una API con funcionalidades de comercio, la consulta de productos también genera un tráfico considerable
- Endpoints con bajo tráfico
 - **POST /api/checkout:** Este endpoint, crucial para la conversión (finalizar una compra), muestra un número de requests notablemente bajo en comparación con otros. Esto podría ser una señal de alerta, sugiriendo:
 - ◇ Una baja tasa de conversión de carritos a compras finalizadas.
 - ◇ Posibles problemas de usabilidad o errores que impiden a los usuarios llegar o completar el proceso de checkout.
 - ◇ Que el periodo de tiempo analizado no sea representativo de picos de venta. Que el periodo de tiempo analizado no sea representativo de picos de venta.
 - **POST /api/cart/add:** Similar al checkout, añadir productos al carrito también tiene un volumen bajo. Esto, junto con el bajo checkout, podría indicar una baja interacción general con el proceso de compra o que los usuarios navegan mucho (GET /api/productos es alto) pero no concretan acciones de compra.
- Podemos observar que las “Request Count E” y “Request Count F” manejan la mayor parte del tráfico y esto es debido a que han sido las pruebas realizadas con mayor carga, por lo tanto, no indicaría una mala configuración del balanceo de carga
- Los endpoints de tipo GET (consulta de información) como /api/pedidos, /api/productos, /api/carrito, /api/menu y /api/login tienden a tener un volumen de requests más alto que los de modificación (POST, PUT, PATCH, DELETE) con las excepciones de POST /api/carrito/vaciar. Esto es un patrón común, donde las operaciones de lectura suelen ser más frecuentes.

4.2. Tiempo medio de respuesta



Esta gráfica de barras representa el tiempo medio de respuesta (en milisegundos) de cada endpoint de la API, desglosado adicionalmente por series denominadas “Average Response Time” a “Average Response Time F”. Como se ha comentado en la gráfica anterior, la serie “Average Response Time A” corresponde con la primera prueba de carga (configurada para 10 usuarios concurrentes, con una rampa de subida de 2 usuarios y una duración total de 1 minuto) y “Average Response Time F” corresponde con la última (configurada para 200 usuarios concurrentes, con una rampa de subida de 50 usuarios y una duración total de 5 minutos). Las series intermedias (B,C,D,E) corresponden con las correspondientes pruebas de cargas intermedias. Al igual que se ha comentado en la gráfica anterior, se ha usado el plugin “Infinity” para insertar el csv correspondiente y después se han seleccionado los campos de “Action” y “Average Response Time” para mostrar los datos de los endpoints y el tiempo medio de respuesta de cada uno.



Se han realizado 6 queries de la misma forma para representar cada prueba de carga y se han hecho dos transformaciones.

- **Convert field type:** Se ha realizado una conversión de tipo sobre el campo “Average Response Time” a tipo numérico ya que por defecto venía en tipo texto
- **Join by field:** Esta transformación lo que nos permite es agrupar todas las queries por un campo. El campo seleccionado para hacer la agrupación es “Action” ya que lo que nos interesa es ver el tiempo medio de respuesta de cada endpoint

Las razones de la elección de esta gráfica son las siguientes:

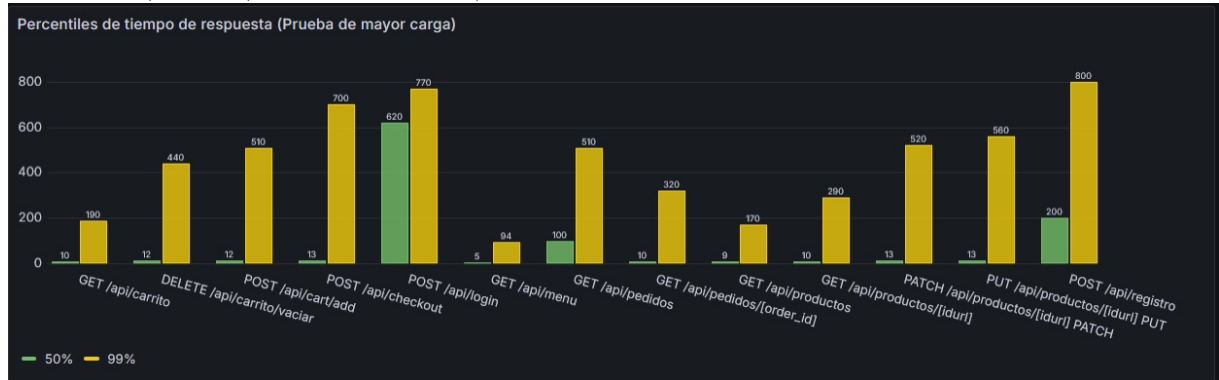
- El tiempo medio de respuesta es una métrica crítica pues impacta directamente en:
 - **Experiencia del usuario:** Tiempos de respuesta elevados generan frustración en el mismo haciendo que en algunos casos se lleve al abandono del servicio. Una respuesta rápida es percibida como un sistema eficiente y de calidad
 - La forma en la que el tiempo medio de respuesta evoluciona al aumentar la carga es un indicativo de la **capacidad del sistema para escalar**. Si los tiempos crecen desproporcionalmente, señala posibles cuellos de botella.
 - Permite identificar qué **operaciones o endpoints** son inherentemente más **lentos** o se degradan más significativamente ante estrés.
- Si se realizan cambios en el código, infraestructura, esta gráfica es esencial para comparar el rendimiento antes y después. Se pueden ejecutar las mismas pruebas y comparar las gráficas para ver si las optimizaciones tuvieron el efecto deseado o para detectar si algún cambio produjo una regresión en el rendimiento de algún endpoint
- Una vez identificado los endpoints problemáticos, se puede dedicar más tiempo al profiling más detallados de esas sesiones para encontrar las causas raíz de esa lentitud.

Al analizar la gráfica podemos sacar las siguiente conclusiones:

- **POST /api/checkout** es un cuello de botella crítico bajo alta carga. Este endpoint muestra una degradación dramática en el tiempo medio de respuesta a medida que aumenta la carga. En la prueba F (la de mayor estrés) podemos observar que el tiempo se dispara a casi 12000ms (12 segundos). Con esto podemos ver que esta operación es el principal punto débil de la aplicación en términos de escalabilidad y rendimiento. En condiciones de alta carga, al experiencia del usuario al realizar un checkout sería inaceptablemente lenta, pudiendo llevar a pérdida de transacciones.
- **POST /api/registro** también sufre bajo carga elevada. Aunque esta degradación no es tan extrema como la del endpoint anterior, el endpoint del registro presenta un aumento significativo en el tiempo de respuesta con las pruebas de mayor carga (D,E y especialmente la F, llegando a unos 2000-2500 ms). Este endpoint merece atención para mejorar la experiencia de nuevos usuarios en momentos de alta demanda.
- La mayoría de los endpoints de lectura (GET) y operaciones simples son resilientes. Endpoints como GET /api/carrito, GET /api/menu, GET /api/productos, e incluso operaciones de escritura simples como POST /api/cart/add o DELETE /api/carrito/vaciar, mantienen unos tiempos medios de respuesta consistentemente bajos y estables a través de todas las pruebas de carga, incluyendo la más exigente (F) Esto indica que esas funcionalidades de la API están bien optimizadas o que son inherentemente más sencillas, manejando bien el aumento de la concurrencia y no presentan una preocupación significativa.
- Se observa un salto más pronunciado en los tiempos medios de respuesta entre las pruebas C,D y se agudiza en F. El sistema parece manejar correctamente las cargas representadas por las pruebas A,B,C. Sin embargo, a partir de la D y más claramente en la E y F, el sistema empieza a tener dificultades para mantener los

tiempos medios de respuesta aceptables y esto indica que el sistema está alcanzando límites de recursos o eficiencia en esas operaciones.

Tras el análisis de la gráfica de tiempos medios de respuesta, resulta igualmente importante examinar la distribución de estos tiempos mediante percentiles, especialmente bajo el escenario de mayor carga (Prueba 6, 200 usuarios). La siguiente gráfica (*ver Figura 4.2, asumiendo que tienes una figura con esa etiqueta*) muestra los percentiles 50 (mediana, P50, en verde) y 99 (P99, en amarillo) para cada endpoint.



El percentil 50 (P50) indica el tiempo de respuesta que experimenta el usuario típico (la mitad de las solicitudes son más rápidas que este valor), mientras que el percentil 99 (P99) refleja la latencia experimentada por el 99 % de las solicitudes, ofreciendo una visión de la experiencia en el peor de los casos (excluyendo el 1 % de valores atípicos más extremos). Una diferencia notable entre P50 y P99 para un mismo endpoint sugiere una variabilidad significativa en los tiempos de respuesta, donde algunos usuarios pueden experimentar demoras considerablemente mayores que la media o la mediana.

Análisis de los Percentiles de Tiempo de Respuesta:

Observando la gráfica de percentiles bajo la prueba de mayor carga, se extraen las siguientes conclusiones:

■ Endpoints con Rendimiento Crítico Identificados por P99:

- **POST /api/registro:** Presenta el peor rendimiento, con un P99 de 800ms y un P50 ya elevado de 200ms. Esto indica que el proceso de registro de nuevos usuarios se degrada severamente bajo alta carga, ofreciendo una mala experiencia incluso para el usuario típico.
- **POST /api/checkout:** Confirma su condición de cuello de botella, con un P99 de 770ms y un P50 de 620ms. Ambos valores son excesivamente altos, lo que significa que la finalización de una compra es consistentemente lenta para la mayoría de los usuarios bajo estrés.
- **PUT /api/productos/[idurl]** y **PATCH /api/productos/[idurl]:** Muestran P99 de 560ms y 520ms respectivamente, con medianas (P50) muy bajas (13ms). Esta gran disparidad sugiere que, si bien la mayoría de las actualizaciones son rápidas, un pequeño porcentaje de ellas sufre demoras significativas, posiblemente debido a bloqueos de base de datos o contención de recursos al escribir.
- **POST /api/cart/add** y **GET /api/pedidos:** Ambos presentan P99 de 510ms, con medianas también bajas (12ms y 10ms). Similar a las actualizaciones de productos, la mayoría de las operaciones son rápidas, pero existen picos de latencia notables.

▪ **Endpoints con Variabilidad Notable (P99 significativamente mayor que P50):**

- DELETE /api/carrito/vaciar: P50 de 12ms frente a un P99 de 440ms.
- GET /api/productos: P50 de 10ms frente a un P99 de 290ms.
- GET /api/carrito: P50 de 10ms frente a un P99 de 190ms.
- GET /api/pedidos/[order_id]: P50 de 9ms frente a un P99 de 170ms. Para estos endpoints, aunque la experiencia típica (P50) es muy buena, la existencia de un P99 considerablemente más alto indica que una porción de los usuarios (aunque minoritaria) experimenta tiempos de respuesta mucho peores. Esto podría deberse a factores como cold starts", recolección de basura, o variaciones en la carga de la base de datos.

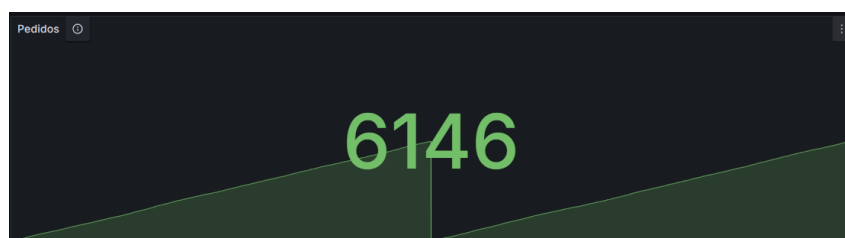
▪ **Endpoints con Rendimiento Generalmente Bueno y Estable:**

- POST /api/login: P50 de 5ms y P99 de 94ms.
- GET /api/menu: P50 de 5ms y P99 de 100ms. Estos endpoints muestran tiempos de respuesta bajos y una diferencia relativamente pequeña entre P50 y P99, indicando un rendimiento consistente incluso bajo la prueba de mayor carga.

En resumen, el análisis de percentiles confirma que los procesos de /api/registro y /api/checkout son los más problemáticos en términos de latencia bajo carga. Adicionalmente, varios endpoints muestran una variabilidad considerable en sus tiempos de respuesta, donde el percentil 99 es significativamente más alto que la mediana. Esta información es crucial, ya que la experiencia del usuario no solo se define por el promedio, sino también por la consistencia y la predictibilidad del rendimiento. Una alta variabilidad, incluso con una buena mediana, puede llevar a una percepción negativa del servicio por parte de aquellos usuarios que experimentan los tiempos de respuesta más lentos. Las optimizaciones futuras deberían enfocarse no solo en reducir los tiempos promedio de los cuellos de botella, sino también en disminuir la dispersión entre los percentiles P50 y P99 para los demás endpoints.

5. Métricas de Modelo de Negocio

5.1. Número de pedidos



Esta gráfica representa el **número total de pedidos hechos a la tienda**. La gráfica es un dato simple porque solo queremos representar un número. El dato se obtiene de la tabla orders, contando el número de filas que tiene la tabla, representando cada una

de ellas un pedido diferente, por lo que la suma total es el número de pedidos realizados hasta el momento.

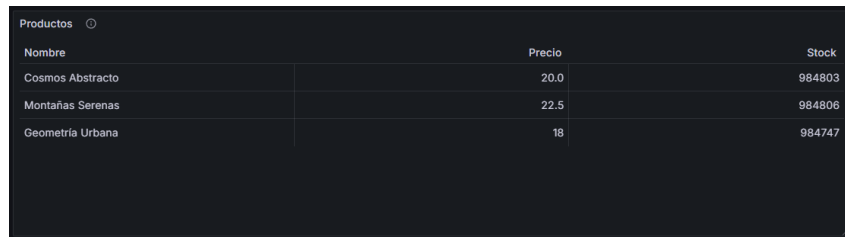
Configuración:

La consulta realizada para obtener los datos es la siguiente:

```
1 SELECT id FROM ArteDB.orders
```

Para que se cuenten las filas, en la opción calculation seleccionamos la opción *count*.

5.2. Productos



Nombre	Precio	Stock
Cosmos Abstracto	20.0	984803
Montañas Serenas	22.5	984806
Geometría Urbana	18	984747

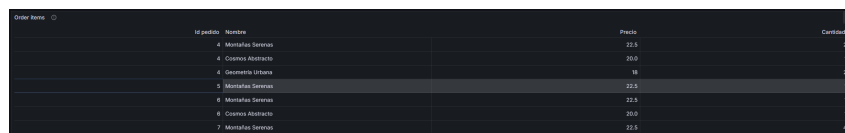
Ésta gráfica representa todos los **productos disponibles en la tienda**. Lo representamos en forma de tabla, de manera que se puede apreciar el nombre de cada producto, su precio y el stock disponible en tienda. Los datos se obtienen de la tabla products.

La consulta realizada para obtener los datos es la siguiente:

```
1 SELECT title AS "Nombre", price AS "Precio", stock AS "Stock"
FROM ArteDB.products
```

Para que se entiendan mejor los datos, hemos puesto alias

5.3. Productos por pedido



Id pedido	Nombre	Precio	Cantidad
4	Montañas Serenas	22.5	2
4	Cosmos Abstracto	20.0	1
4	Geometría Urbana	18	1
5	Montañas Serenas	22.5	2
6	Montañas Serenas	22.5	1
6	Cosmos Abstracto	20.0	1
7	Montañas Serenas	22.5	4

Ésta gráfica representa los **productos totales de cada pedido**. Lo representamos en forma de tabla, de manera que se visualiza el id del pedido, y a la derecha el producto que se ha ordenado, su precio y la cantidad de éste. Como por cada pedido pueden ordenarse varios productos, se puede ver que en ocasiones aparece el id de pedido repetido. Los datos los obtenemos de la tabla order_items.

La consulta realizada para obtener los datos es la siguiente:

```
1 SELECT oi.order_id AS "Id pedido", p.title as "Nombre", oi.
price_at_purchase AS "Precio", oi.quantity AS "Cantidad" FROM
ArteDB.order_items oi,ArteDB.products p where p.id = oi.
product_id order by oi.order_id
```

Para que se entiendan mejor los datos, hemos puesto alias, y como no guardamos en ésta tabla el nombre de los productos sino el id, una referencia a ese nombre.

5.4. Productos por carrito

	Id usuario	Nombre	Cantidad
	5	Cosmos Abstracto	2
	5	Montañas Serenas	1
	6	Cosmos Abstracto	1
	6	Montañas Serenas	1
	45	Montañas Serenas	3
	45	Geometría Urbana	1

Ésta gráfica representa los **productos totales añadidos a cada carrito**. Lo representamos en forma de tabla, de manera que se visualiza el id del usuario, y a la derecha el producto que se ha añadido y la cantidad de éste. Como por cada carrito pueden añadirse varios productos, se puede ver que en ocasiones aparece el id de pedido repetido. Los datos los obtenemos de la tabla `cart_items`.

La consulta realizada para obtener los datos es la siguiente:

```
1 select ci.user_id as "Id usuario",p.title as "Nombre",ci.  
quantity as "Cantidad" from ArteDB.cart_items ci, ArteDB.  
products p where ci.product_id = p.id order by ci.user_id
```

Para que se entiendan mejor los datos, hemos puesto alias, y como no guardamos en ésta tabla el nombre de los productos sino el id, una referencia a ese nombre.

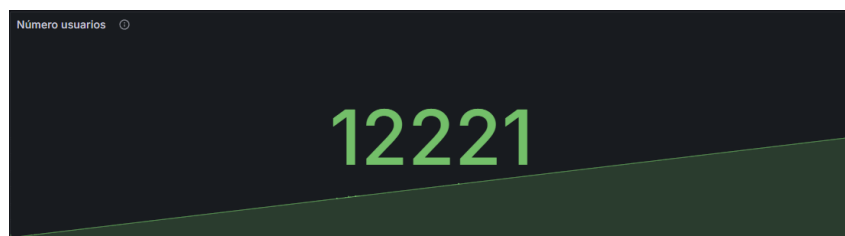
5.5. Número de usuarios

Ésta gráfica representa el **número total de usuarios registrados en la tienda**. La gráfica es un dato simple porque solo queremos representar un número. El dato se obtiene de la tabla `users`, y lo que hace es contar el número de filas que tiene la tabla, representando cada una de ellas un usuario diferente, por lo que la suma total es el número de usuarios logueados hasta el momento.

La consulta realizada para obtener los datos es la siguiente:

```
1 SELECT id FROM ArteDB.users
```

Para que se cuenten las filas, en la opción calculation seleccionamos la opción *count*.



6. Conclusiones

La realización de esta práctica ha permitido abordar los objetivos fundamentales planteados en el enunciado, centrados en la **evaluación del rendimiento de una aplicación web mediante pruebas de carga** y la **monitorización de su comportamiento bajo diferentes niveles de estrés**. A través del desarrollo de la tienda online 'Arte Visual' y la aplicación sistemática de herramientas como Locust y Grafana, se han obtenido resultados sobre la capacidad de respuesta, escalabilidad y posibles cuellos de botella del sistema implementado.

Uno de los objetivos primordiales era la **monitorización de su comportamiento bajo diferentes niveles de estrés** creación de una aplicación web funcional que sirviera de base para las pruebas. Este objetivo se cumplió mediante el desarrollo de la tienda 'Arte Visual' utilizando el framework Flask en Python. Se implementó una API RESTful completa, con endpoints dedicados para la autenticación de usuarios, la gestión del catálogo de productos, la información del menú, las operaciones del carrito de compras y el procesamiento de pedidos, incluyendo el retardo simulado requerido en el proceso de checkout. Adicionalmente, se desarrollaron plantillas HTML y CSS para proporcionar una interfaz de usuario básica, permitiendo la interacción manual con la aplicación y facilitando la comprensión de los flujos de usuario que posteriormente serían simulados.

El segundo objetivo clave consistió en **instalar y utilizar Locust para generar carga** sobre la aplicación desplegada en la máquina virtual. Este objetivo se alcanzó plenamente mediante la creación de un `locustfile.py` que definió el comportamiento de usuarios virtuales. Estos usuarios simulados realizaron un conjunto diverso de acciones, incluyendo el inicio de sesión, la navegación por el catálogo, la adición de productos al carrito, la visualización del carrito y la finalización de compras, así como la actualización de productos. La ponderación de estas tareas se ajustó para reflejar patrones de uso realistas, permitiendo someter a la API a una carga representativa.

Paralelamente, se abordó el objetivo de **monitorizar el rendimiento del servidor y de la aplicación utilizando Grafana**. Se configuraron y visualizaron métricas clave del servidor (uso de CPU, memoria, E/S de disco y tráfico de red), las cuales se complementaron con los datos generados por Locust (peticiones por segundo, tiempos de respuesta, tasa de fallos). De especial relevancia fue la integración de métricas de negocio directamente extraídas de la base de datos de la aplicación, como el número total de pedidos, el número de usuarios registrados, y la composición de los carritos y pedidos. Esta monitorización integral permitió no solo observar el impacto de la carga en los recursos del sistema, sino también correlacionar el rendimiento técnico con indicadores funcionales de la tienda.

El **análisis de los resultados** obtenidos a través de las sucesivas pruebas de carga con un número creciente de usuarios virtuales (desde 10 hasta 200) reveló el comportamiento del sistema bajo estrés. Se observó que la aplicación Arte Visual mantenía un rendimiento estable y tiempos de respuesta bajos con cargas de hasta aproximadamente 50 usuarios concurrentes. A partir de este punto, con 100 usuarios o más, comenzaron a evidenciarse signos de saturación, principalmente en el uso de la CPU del servidor, que alcanzaba picos superiores al 80-90 %. Esto se tradujo en un incremento notable en los tiempos de respuesta promedio y, de manera crítica, en los percentiles 95 de latencia, especialmente

para operaciones complejas como el checkout y el registro de nuevos usuarios. Con cargas de 150-200 usuarios, el sistema experimentó una degradación severa del rendimiento, con tiempos de respuesta inaceptables y un aumento significativo en la tasa de fallos de las peticiones, indicando que se había superado la capacidad operativa del sistema en su configuración actual.

Se **identificó** que los **endpoints** que implican múltiples operaciones de escritura en la base de datos y lógica de negocio más compleja, como `POST /api/checkout` y `POST /api/registro`, fueron los primeros en mostrar degradación y se convirtieron en los principales cuellos de botella bajo alta carga. Por el contrario, los endpoints de lectura (**GET**) y operaciones más simples del carrito mantuvieron un rendimiento más resiliente. La inclusión de manejo de deadlocks en el endpoint de checkout y la ordenación de actualizaciones de stock fueron decisiones de implementación que buscaron mitigar problemas de concurrencia, aunque la carga extrema eventualmente superó estas optimizaciones a nivel de aplicación.

En conclusión, esta práctica ha permitido validar la metodología de pruebas de carga como una herramienta esencial para la evaluación del rendimiento de software. Se cumplieron los objetivos de desarrollar una aplicación, someterla a carga simulada con Locust y monitorizar su comportamiento con Grafana. Los resultados obtenidos proporcionan una base clara para futuras optimizaciones, como la mejora de consultas SQL, la posible implementación de mecanismos de caché, el ajuste de la configuración de la base de datos o, en un escenario de mayor demanda, la consideración de escalar la infraestructura. El proceso ha subrayado la importancia de un diseño de API eficiente y la necesidad de anticipar el comportamiento del sistema bajo condiciones de alta concurrencia para garantizar la calidad y la fiabilidad de las aplicaciones web. ...

7. Referencias

Referencias

- [1] Locust.io. (s.f.). *Locust - An open source load testing tool*. Recuperado de <https://locust.io/>
- [2] Pallets Projects. (s.f.). *Flask Documentation*. Recuperado de <https://flask.palletsprojects.com/>
- [3] Python Software Foundation. (s.f.). *Python 3 Documentation*. Recuperado de <https://docs.python.org/3/>
- [4] Grafana Labs. (s.f.). *Grafana Documentation*. Recuperado de <https://grafana.com/docs/grafana/latest/>
- [5] Prometheus.io. (s.f.). *Prometheus - Monitoring system & time series database*. Recuperado de <https://prometheus.io/docs/introduction/overview/>
- [6] MariaDB Foundation. (s.f.). *MariaDB Server Documentation*. Recuperado de <https://mariadb.com/kb/en/documentation/>
- [7] Oracle Corporation. (s.f.). *MySQL Documentation*. Recuperado de <https://dev.mysql.com/doc/>
- [8] Pallets Projects. (s.f.). *Jinja2 Documentation*. Recuperado de <https://jinja.palletsprojects.com/>
- [9] Pallets Projects. (s.f.). *Werkzeug Security Helpers*. Recuperado de <https://werkzeug.palletsprojects.com/en/latest/utils/#security-helpers>
- [10] Wikipedia. (s.f.). *Load testing*. Recuperado de https://en.wikipedia.org/wiki/Load_testing
- [11] MariaDB Knowledge Base. (s.f.). *Deadlocks*. Recuperado de <https://mariadb.com/kb/en/deadlocks/>
- [12] Mozilla Developer Network (MDN). (s.f.). *HTTP response status codes*. Recuperado de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>